

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

До захисту допущено:
В.о. завідувача кафедри

_____ Артем ВОЛОКИТА
(підпис)
_____ 2026 р.
(число) (місяць)

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Комп'ютерні системи та мережі»
спеціальність 123 — «Комп'ютерна інженерія»

**КРОСПЛАТФОРМНА КЛІЄНТ-СЕРВЕРНА СИСТЕМА
АГРЕГАЦІЇ ТА ОБРОБКИ МЕДИКО-БІОЛОГІЧНИХ ДАНИХ
З ВИКОРИСТАННЯМ ХМАРНИХ ТЕХНОЛОГІЙ**

Виконав

Льоскін Іван Вадимович, студент 4 курсу, група ІО-25 _____
(підпис)

Керівник

Кобилюк Андрій Григорович, посада, науковий ступінь _____
(підпис)

Консультант

Сімоненко Валерій Павлович, доктор технічних наук, професор _____
(підпис)

Рецензент

Писаренко Андрій Володимирович, кандидат наук, доцент _____
(підпис)

Засвідчую, що у цьому дипломному проєкті немає запозичень
з праць інших авторів без відповідних посилань

_____ **Льоскін І. В.**
(підпис)

Київ — 2026 р.

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Освітньо-кваліфікаційний рівень: Бакалавр

Освітньо-професійна програма: «Комп'ютерні системи та мережі»

Спеціальність: 123 — «Комп'ютерна інженерія»

ЗАТВЕРДЖУЮ

В.о. завідувача кафедри

Артем ВОЛОКИТА

_____ (підпис)

_____ 2026 р.

_____ (число)

_____ (місяць)

ЗАВДАННЯ

на бакалаврський дипломний проєкт

Льоскіну Івану Вадимовичу

1. Тема проєкту: Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій
керівник Кобилюк Андрій Григорович, посада, науковий ступінь
затверджено наказом по університету від 5 червня 2026 року №1212-с
2. Дата здачі закінченого проєкту 5 червня 2026 року
(число, місяць, рік)
3. Вихідні дані для проєкту: технічна документація, теоретичні та статистичні дані, патенти на винахід, наукові публікації.
4. Зміст розрахунково-пояснювальної записки: опис предметної області і напрямків дослідження, аналіз та характеристика об'єкту досліджень, аналіз існуючих рішень, опис архітектури системи.

5. Перелік графічного матеріалу: принципова, структурна та функціональна схеми системи.

6. Консультанти розділів проєкту

№	Розділ	Консультант (прізвище та ініціали)	Підпис, дата	
			завдання видав	завдання прийняв
1	Нормоконтроль	Сімоненко В. П.		

7. Дата видачі завдання 10 травня 2026 року
(число, місяць, рік)

Календарний план

№	Назва етапу виконання дипломного проєкту	Термін виконання	Примітка
1	Затвердження теми проєкту	06.04.2026–07.06.2026	
2	Вивчення та аналіз завдання	20.04.2026–26.04.2026	
3	Розробка архітектури та загальної структури системи	27.04.2026–03.05.2026	
4	Програмна реалізація системи	04.05.2026–17.05.2026	
5	Оформлення пояснювальної записки	18.05.2026–07.06.2026	
6	Передзахист		
7	Захист		

Виконавець _____ Льоскін Іван Вадимович
(підпис)

Керівник _____ Кобилюк Андрій Григорович
(підпис)

АНОТАЦІЯ

У даній дипломній роботі проведено огляд та аналіз існуючих мобільних застосунків для відстеження розвитку немовляти, досліджено їх переваги та недоліки. На основі аналізу предметної області та потреб користувачів сформульовано вимоги до системи, обрано технологічний стек і спроектовано архітектуру кросплатформного мобільного застосунку. Розроблено реляційну схему бази даних із захистом на рівні рядків (Row-Level Security), що забезпечує ізоляцію даних між користувачами, та визначено ключові модулі системи.

В результаті розроблено кросплатформний мобільний застосунок для платформ iOS та Android, що поєднує щоденник активностей немовляти, моніторинг антропометричних показників відповідно до стандартів ВООЗ, управління вакцинаціями та досягненнями, а також ШІ-асистента на основі технології Retrieval-Augmented Generation. Застосунок реалізовано з використанням React Native та Expo; серверну частину побудовано на Supabase із PostgreSQL, pgvector та Row-Level Security. ШІ-асистента інтегровано через Groq API з векторним пошуком у базі педіатричних знань. Розгортання здійснено на хмарній інфраструктурі з підтримкою синхронізації між пристроями в реальному часі. Проведено тестування системи на рівнях юніт-тестів, тестів бази даних і наскрізних тестів.

Ключові слова: мобільний застосунок, React Native, Expo, Supabase, pgvector, RAG, ШІ-асистент, відстеження розвитку немовляти, ВООЗ.

ANNOTATION

In this bachelor's thesis, a review and analysis of existing mobile applications for infant development tracking was conducted, and their advantages and shortcomings were examined. Based on the subject domain analysis and user needs, system requirements were formulated, a technology stack was selected, and the architecture of a cross-platform mobile application was designed. A relational database schema with Row-Level Security was developed to ensure data isolation between users, and the key system modules were defined.

As a result, a cross-platform mobile application for iOS and Android platforms was developed, combining an infant activity journal, monitoring of anthropometric indicators in accordance with WHO standards, management of vaccinations and developmental milestones, and an AI assistant based on Retrieval-Augmented Generation technology. The application was implemented using React Native and Expo; the backend was built on Supabase with PostgreSQL, pgvector and Row-Level Security. The AI assistant was integrated via the Groq API with vector search in a paediatric knowledge base. The system was deployed on cloud infrastructure with support for real-time synchronisation between devices. System testing was conducted at unit, database and end-to-end levels.

Keywords: mobile application, React Native, Expo, Supabase, pgvector, RAG, AI assistant, infant development tracking, WHO.

№ рядка	Формат	Значення	Найменування	Кіл. листів	№ екземпляра	Додаток
1			Документація загальна			
2			Знову розроблена			
3						
4	A4	ІАЛЦ.467200.002 ТЗ	Кросплатформна клієнт-серверна			
5			система агрегації та обробки			
6			медико-біологічних даних з			
			використанням хмарних технологій			
7			Технічне завдання			
8						
9	A4	ІАЛЦ.467200.003 ПЗ	Кросплатформна клієнт-серверна			
10			система агрегації та обробки			
11			медико-біологічних даних з			
12			використанням хмарних технологій			
			Пояснювальна записка			
13						
14	A4	ІАЛЦ.467200.004 Д1	Кросплатформна клієнт-серверна			
			система агрегації та обробки			
15			медико-біологічних даних з			
16			використанням хмарних технологій			
17			Принципова схема			
18						
19	A1	ІАЛЦ.467200.005 Д2	Кросплатформна клієнт-серверна			
20			система агрегації та обробки			
21			медико-біологічних даних з			
22			використанням хмарних технологій			
23			Функціональна схема			
24						
25	A1	ІАЛЦ.467200.006 Д3	Кросплатформна клієнт-серверна			
26			система агрегації та обробки			
27			медико-біологічних даних з			
28			використанням хмарних технологій			
29			Структурна схема			
30						
31	A1	ІАЛЦ.467200.006 Д4	Кросплатформна клієнт-серверна			
32			система агрегації та обробки			
33			медико-біологічних даних з			
34			використанням хмарних технологій			
35			Текст програмного коду			
36						
37						
38						
39						

					ІАЛЦ.467200.001 ОА						
Зм.	Арк.	№ докум.	Підп.	Дата	Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій Опис альбому	Лім.	Ар.	Аркушів			
Розроб.	Льоскін І. В.										
Перевір.	Кобилук А. Г.						1	1			
Реценз.						НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-25					
Н. контр.											
Затверд.											

ТЕХНІЧНЕ ЗАВДАННЯ ДО ДИПЛОМНОГО ПРОЄКТУ

на тему: *«Кросплатформна клієнт-серверна система агрегації та обробки
медико-біологічних даних з використанням хмарних технологій»*

Київ – 2026

ЗМІСТ

1 НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ.....	2
2 ПІДСТАВИ ДЛЯ РОЗРОБКИ	2
3 МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ	2
4 ДЖЕРЕЛА РОЗРОБКИ.....	3
5 ТЕХНІЧНІ ВИМОГИ	3
5.1. Вимоги до розробленого продукту	3
5.2. Вимоги до програмного забезпечення	4
5.3. Вимоги до апаратної частини.....	4
6 ЕТАПИ РОЗРОБКИ.....	4

					ІАЛЦ.467200.002 ТЗ			
Зм.	Лист	№ докум.	Підп.	Дата				
Розробив	Льоскін І. В.				Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій Технічне завдання	Лит.	Аркуш	Аркушів
Перевірив	Кобилюк А. Г.						1	4
Н. контр.	Сімоненко В. П.					НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ 10-25		
Затвердив	Волокита А. М.							

1 НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ

Це технічне завдання стосується розробки кросплатформного мобільного застосунку для відстеження розвитку немовляти. Застосунок призначений для батьків та опікунів, які потребують зручного інструменту для моніторингу активностей дитини, контролю антропометричних показників відповідно до стандартів ВООЗ, управління вакцинаціями та досягненнями, а також отримання персоналізованих педіатричних рекомендацій. Система розрахована на використання в побутових умовах на мобільних пристроях під управлінням iOS та Android.

2 ПІДСТАВИ ДЛЯ РОЗРОБКИ

Підставою для розробки цього програмного продукту є виконання вимог кваліфікаційно-освітнього рівня «бакалавр» зі спеціальності «Комп'ютерна інженерія», затвердженого відповідними освітніми та науковими установами Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

3 МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ

Метою розробки є створення зручного та функціонального кросплатформного мобільного застосунку, який допоможе батькам систематично відстежувати розвиток немовляти в перші роки життя.

Застосунок призначений для ведення щоденника активностей дитини (годування, сон, підгузки), моніторингу росту та ваги з порівнянням до стандартів ВООЗ, відстеження вакцинацій і вікових досягнень, а також отримання відповідей на педіатричні запитання через ШІ-асистента на основі перевіреної бази знань. Система забезпечує синхронізацію даних між пристроями в реальному часі, що дозволяє обом батькам мати актуальну

					ІАЛЦ.467200.002 ТЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		2

інформацію одночасно.

4 ДЖЕРЕЛА РОЗРОБКИ

Джерелами розробки даного дипломного проєкту є офіційна технічна документація до використовуваних технологій (React Native, Expo, Supabase, Groq API), стандарти ВООЗ щодо показників росту та розвитку дітей, наукові публікації у сфері мобільної розробки та штучного інтелекту, відкриті бази педіатричних знань, а також статті та матеріали в мережі Інтернет за тематикою проєкту.

5 ТЕХНІЧНІ ВИМОГИ

5.1. Вимоги до розробленого продукту

Розроблена система повинна задовольняти наступні вимоги:

- кросплатформна підтримка iOS 16.0 і новіших та Android 10 (API 29) і новіших версій;
- автентифікація користувачів та ізоляція даних на рівні рядків бази даних (Row-Level Security);
- синхронізація даних між пристроями в реальному часі через Supabase Realtime;
- відображення графіків зростання та порівняння з еталонними кривими ВООЗ;
- ШІ-асистент на основі Retrieval-Augmented Generation з векторним пошуком у педіатричній базі знань;
- підтримка кількох профілів дітей в рамках одного облікового запису;
- зрозумілий та інтуїтивний інтерфейс, що не вимагає попередньої

технічної підготовки.

5.2. Вимоги до програмного забезпечення

- операційна система: iOS 16.0+ або Android 10+;
- активне підключення до мережі Інтернет для синхронізації та роботи ШІ-асистента;
- для розробки: Node.js 18+, Expo CLI, Git.

5.3. Вимоги до апаратної частини

- смартфон з оперативною пам'яттю не менше 3 ГБ;
- наявність підключення до мережі Інтернет (Wi-Fi або мобільний інтернет).

6 ЕТАПИ РОЗРОБКИ

Назва етапу виконання	Термін виконання
Затвердження теми роботи	
Вивчення та аналіз завдання	
Розробка архітектури та загальної структури системи	
Розробка структур окремих частин системи	
Програмна реалізація системи	
Виправлення помилок	
Оформлення пояснювальної записки	

ПОЯСНЮВАЛЬНА ЗАПИСКА ДО ДИПЛОМНОГО ПРОЄКТУ

на тему: *«Кросплатформна клієнт-серверна система агрегації та обробки
медико-біологічних даних з використанням хмарних технологій»*

Київ – 2026

ЗМІСТ

ПЕРЕЛІК СКОРОЧЕНЬ.....	3
ВСТУП	4
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД ІСНУЮЧИХ РІШЕНЬ.....	6
1.1 Актуальність та доцільність розроблення.....	6
1.2 Огляд та аналіз існуючих рішень.....	7
1.2.1 Sprout Baby Tracker	7
1.2.2 Huckleberry	8
1.2.3 Glow Baby.....	9
1.3 Аналіз функціональності та вимог до системи	11
1.3.1 Порівняльна характеристика розглянутих рішень.....	11
1.3.2 Вимоги до реалізації застосунку	13
ВИСНОВОК ДО РОЗДІЛУ 1	15
РОЗДІЛ 2. ОГЛЯД ТА ВИБІР ТЕХНОЛОГІЙ ДЛЯ ПРОЕКТУВАННЯ СИСТЕМИ.....	16
2.1 Загальні принципи побудови кросплатформних мобільних застосунків.....	16
2.1.1 Клієнт-серверна архітектура мобільних застосунків.....	16
2.1.2 Основні компоненти мобільного застосунку.....	17
2.2 Огляд технологій для реалізації застосунку.....	18
2.2.1 Технології клієнтської частини.....	18
2.2.2 Хмарна платформа та серверна частина	19
2.2.3 Характеристика системи управління базами даних.....	20
2.2.4 Управління станом, локалізація та допоміжні бібліотеки ...	20
2.3 Механізми автентифікації та захисту даних.....	21
2.3.1 Row-Level Security та ізоляція даних користувачів	21
2.3.2 Безпека мобільного застосунку	22
2.4 Стандарти ВООЗ та педіатричний моніторинг	22
2.5 Інтеграція штучного інтелекту на основі RAG	23
2.5.1 Поняття векторного пошуку та RAG-архітектури.....	23
ВИСНОВОК ДО РОЗДІЛУ 2	25

					ІАЛЦ.467200.003 ПЗ			
Зм.	Лист	№ докум.	Підп.	Дата	Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій Пояснювальна записка	Лит.	Аркуш	Аркушів
Розробив	Льоскін І. В.						1	75
Перевірів	Кобилюк А. Г.							
Н. контр.	Сімоненко В. П.					НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-25		
Затвердив	Волокита А. М.							

РОЗДІЛ 3. РЕАЛІЗАЦІЯ СИСТЕМИ	27
3.1 Архітектура системи.....	27
3.1.1 Загальна архітектурна схема застосунку.....	27
3.1.2 Архітектурні особливості клієнтської частини	28
3.1.3 Архітектура хмарної серверної частини	30
3.1.4 Архітектура бази даних.....	31
3.2 Реалізація серверної частини	33
3.2.1 Проєктування та реалізація бази даних.....	33
3.2.2 Реалізація Row-Level Security.....	34
3.2.3 Реалізація Edge Function та ШІ-асистента.....	36
3.3 Реалізація клієнтської частини	38
3.3.1 Організація структури модулів	38
3.3.2 Моніторинг помилок та безпека середовища виконання	41
3.3.3 Навігаційна структура та управління станом	42
3.3.4 Зберігання файлів та синхронізація в реальному часі	43
3.3.5 Реалізація клієнтської автентифікації	44
3.3.6 Реалізація ключових екранів та компонентів	46
3.3.7 Робота з датами та часом.....	53
3.3.8 Реалізація алгоритмів обробки медичних даних	54
ВИСНОВОК ДО РОЗДІЛУ 3	57
РОЗДІЛ 4. ТЕСТУВАННЯ ТА АНАЛІЗ РОЗРОБЛЕНОГО ПРОЄКТУ	59
4.1 Демонстрація роботи та функціональне тестування застосунку...	59
4.1.1 Реєстрація та автентифікація користувача	59
4.1.2 Додавання та відстеження активностей	60
4.1.3 Моніторинг антропометричних показників	61
4.1.4 Консультації ШІ-асистента.....	62
4.2 Тестування ключових функцій системи.....	62
4.2.1 Модульне тестування бізнес-логіки.....	62
4.2.2 Тестування ШІ-асистента.....	64
4.2.3 Перевірка безпеки та ізоляції даних	65
4.2.4 Тестування синхронізації між пристроями	66
4.3 Аналіз результатів тестування	67
4.4 Розгортання та налаштування застосунку	68
ВИСНОВОК ДО РОЗДІЛУ 4	70
ВИСНОВКИ.....	71
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	72

ПЕРЕЛІК СКОРОЧЕНЬ

API	(Application Programming Interface) прикладний програмний інтерфейс
ER	(Entity-Relationship) сутність-зв'язок
HTTPS	(HyperText Transfer Protocol Secure) захищений протокол передачі гіпертексту
JWT	(JSON Web Token) веб-токен JSON
LLM	(Large Language Model) велика мовна модель
LPU	(Language Processing Unit) процесор обробки мови
OAuth	(Open Authorization) відкритий протокол авторизації
RAG	(Retrieval-Augmented Generation) генерація з доповненням пошуком
REST	(Representational State Transfer) передача репрезентативного стану
RLS	(Row-Level Security) безпека на рівні рядків
SDK	(Software Development Kit) набір інструментів розробника
SQL	(Structured Query Language) мова структурованих запитів
UI	(User Interface) інтерфейс користувача
URL	(Uniform Resource Locator) уніфікований локатор ресурсу
БД	база даних
ВООЗ	Всесвітня організація охорони здоров'я
СУБД	система управління базами даних
ШІ	штучний інтелект

ВСТУП

У сучасному суспільстві батьки та опікуни щодня стикаються з необхідністю систематичного відстеження медико-біологічних показників новонародженої дитини: режиму годування та сну, динаміки маси тіла й зросту, графіка вакцинацій і вікових досягнень. ВООЗ наголошує, що регулярний моніторинг фізичного розвитку протягом перших двох років життя є ключовим інструментом раннього виявлення відхилень і своєчасного медичного втручання. Попри широкий вибір мобільних застосунків для моніторингу немовляти, жоден із них не поєднує в собі повноцінного антропометричного моніторингу відповідно до стандартів ВООЗ, синхронізації між пристроями в реальному часі та інтелектуального асистента з перевіреною педіатричною базою знань. Це зумовлює актуальність розроблення комплексної кросплатформної системи, яка усуне зазначені недоліки.

Метою дипломного проєкту є розроблення кросплатформної клієнт-серверної системи агрегації та обробки медико-біологічних даних з використанням хмарних технологій — мобільного застосунку для відстеження розвитку немовляти на платформах iOS та Android. Об'єктом дослідження є процеси агрегації, зберігання та аналізу медико-біологічних даних про розвиток немовляти. Предметом дослідження є методи та інструменти розроблення кросплатформних мобільних застосунків із застосуванням хмарних технологій і ШІ.

У процесі виконання роботи буде проведено аналіз предметної області та огляд існуючих аналогів, сформульовано вимоги до системи. Також буде здійснено огляд та обґрунтування вибору технологічного стеку: React Native та Expo для клієнтської частини, Supabase з PostgreSQL для хмарного бекенду, Groq API для ШІ-асистента. Наступні етапи передбачають

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		4

проектування реляційної схеми бази даних із захистом на рівні рядків (RLS), реалізацію всіх модулів застосунку, інтеграцію ІІІ-асистента на основі архітектури RAG з векторним пошуком, а також тестування системи на рівнях юніт-тестів, тестів бази даних і наскрізних тестів.

Розроблений застосунок є актуальним практичним рішенням для широкого кола користувачів — від молодих батьків, які вперше стикаються з необхідністю ведення медичних записів дитини, до сімей, де обидва батьки потребують спільного доступу до актуальних даних у режимі реального часу. Застосунок розгорнуто на хмарній інфраструктурі та готовий до використання на обох платформах.

РОЗДІЛ 1

АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД ІСНУЮЧИХ РІШЕНЬ

1.1 Актуальність та доцільність розроблення

За даними ВООЗ, перші два роки життя дитини є найбільш інтенсивним періодом фізичного розвитку. Регулярний моніторинг маси тіла, зросту та окружності голови дозволяє педіатрам вчасно виявляти відхилення від вікових норм і призначати корекційні заходи на ранніх стадіях. Паралельно батьки щодня фіксують режим годування та сну, стан здоров'я, вакцинації та вікові досягнення — дані, що формують повноцінну картину розвитку дитини і є незамінними під час лікарських консультацій.

Попри важливість систематичного ведення таких записів, більшість батьків і сьогодні покладаються на паперові щоденники або стандартні нотатки смартфона. Такий підхід має суттєві обмеження: дані легко втрачаються, їх неможливо автоматично перетворити на динамічні графіки, а ведення записів одночасно двома батьками з різних пристроїв призводить до неузгодженостей і втрати актуальності даних. Крім того, паперові записи не дозволяють зіставити показники дитини зі стандартними перцентильними кривими, що унеможлиблює самостійну оцінку динаміки розвитку між лікарськими візитами.

У відповідь на ці потреби ринок мобільних застосунків для батьків активно розвивається. Проте, як показує аналіз наявних рішень, переважна більшість із них охоплює лише окремі аспекти моніторингу і не забезпечує комплексного підходу. Жоден із популярних застосунків не поєднує повноцінного антропометричного моніторингу відповідно до стандартів

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		6

ВООЗ, синхронізації між пристроями в реальному часі та інтелектуального асистента з перевіреною педіатричною базою знань — що визначає практичну актуальність розроблення такої системи.

1.2 Огляд та аналіз існуючих рішень

На ринку мобільних застосунків для батьків існує декілька поширених рішень для відстеження розвитку немовляти. Їх функціональність варіюється від базового журналювання активностей до комплексного відстеження сну з аналітикою. Було розглянуто три найбільш популярних застосунки: Sprout Baby Tracker, Huckleberry та Glow Baby. Аналіз цих рішень дозволив виявити їхні ключові переваги та недоліки, а також визначити напрями для вдосконалення у розроблюваній системі.

1.2.1 Sprout Baby Tracker

Sprout Baby Tracker — один із найстаріших і найвідоміших застосунків для відстеження немовляти, доступний на платформах iOS та Android. Застосунок орієнтований на ведення повноцінного журналу активностей дитини з перших днів життя і дозволяє фіксувати годування (грудне молоко та суміш), епізоди сну, зміни підгузників, вимірювання маси тіла та зросту, а також переглядати хронологію подій у вигляді стрічки. Передбачена підтримка декількох профілів дітей в одному обліковому записі, що зручно для сімей із двома і більше дітьми.

До переваг Sprout Baby Tracker належать зрілий і зручний інтерфейс, широка функціональність журналювання та надійна робота без підключення до мережі. Графіки зростання дозволяють відстежувати динаміку антропометричних показників, однак порівняння з перцентильними кривими ВООЗ безпосередньо в застосунку відсутнє. Синхронізація між пристроями реалізована через хмарне резервне копіювання і не відбувається

в реальному часі, що ускладнює спільне ведення записів двома батьками. ІІІ-асистент або педіатрична база знань у застосунку відсутні.

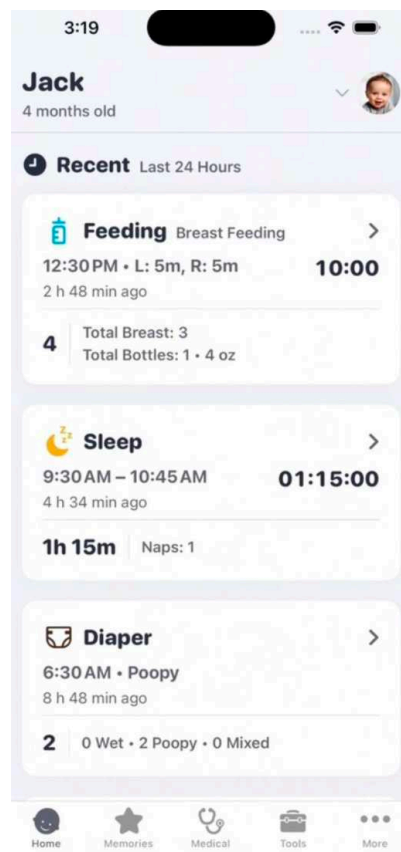


Рисунок 1.1 – Інтерфейс застосунку Sprout Baby Tracker

1.2.2 Huckleberry

Huckleberry — застосунок, орієнтований передусім на аналіз сну немовляти. Його ключова функція — алгоритм SweetSpot, який на основі накопичених даних про денний сон і нічний відпочинок пропонує оптимальний час для вкладання дитини. Застосунок збирає дані про тривалість і якість сну, фіксує годування та дозволяє вести загальні нотатки. Преміальна версія надає доступ до персоналізованих консультацій із сертифікованими фахівцями з дитячого сну.

Перевагою Huckleberry є глибока аналітика сну та наявність консультаційного сервісу, що виділяє його серед аналогів. Водночас застосунок має суттєві обмеження: повноцінний модуль антропометричного

моніторингу з перцентильними кривими ВООЗ відсутній, функціональність безкоштовної версії значно обмежена, а покрокового журналу активностей, який охоплював би всі аспекти догляду за немовлям, немає. ІІІ-асистент на основі бази педіатричних знань у застосунку не реалізований.

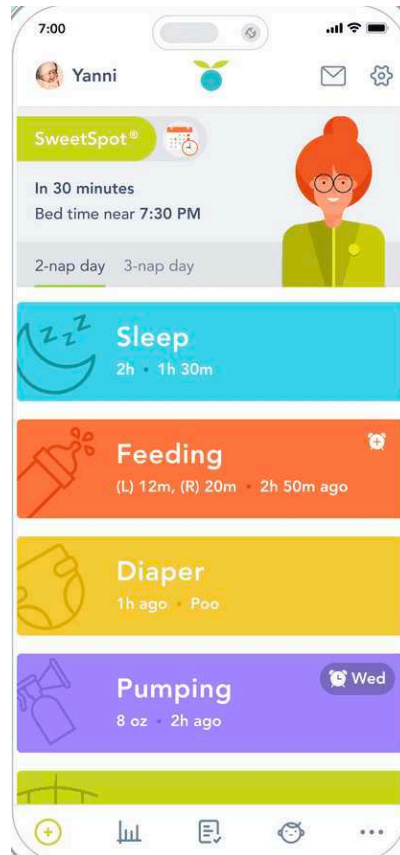


Рисунок 1.2 – Інтерфейс застосунку Huckleberry

1.2.3 Glow Baby

Glow Baby — комплексний трекер розвитку дитини від компанії Glow Inc., доступний на iOS та Android. Застосунок охоплює журнал активностей (годування, сон, підгузники, купання), відстеження вікових досягнень за категоріями (моторика, мова, соціальний розвиток), базові графіки зростання та широку базу статей про догляд за немовлятами. Передбачена функція спільного доступу, що дозволяє обом батькам переглядати та додавати записи.

Glow Baby вирізняється широтою контенту, системою вікових підказок і наявністю спільного доступу для батьків. Проте порівняння антропометричних показників із перцентильними кривими ВООЗ реалізоване спрощено і не відображає повний набір стандартів. Статті носять загальний інформаційний характер без прив'язки до конкретних даних дитини. Персоналізованого ШІ-асистента, здатного відповідати на педіатричні запитання на основі перевіреної бази знань, у застосунку немає. Синхронізація між пристроями в режимі реального часу також не забезпечена.

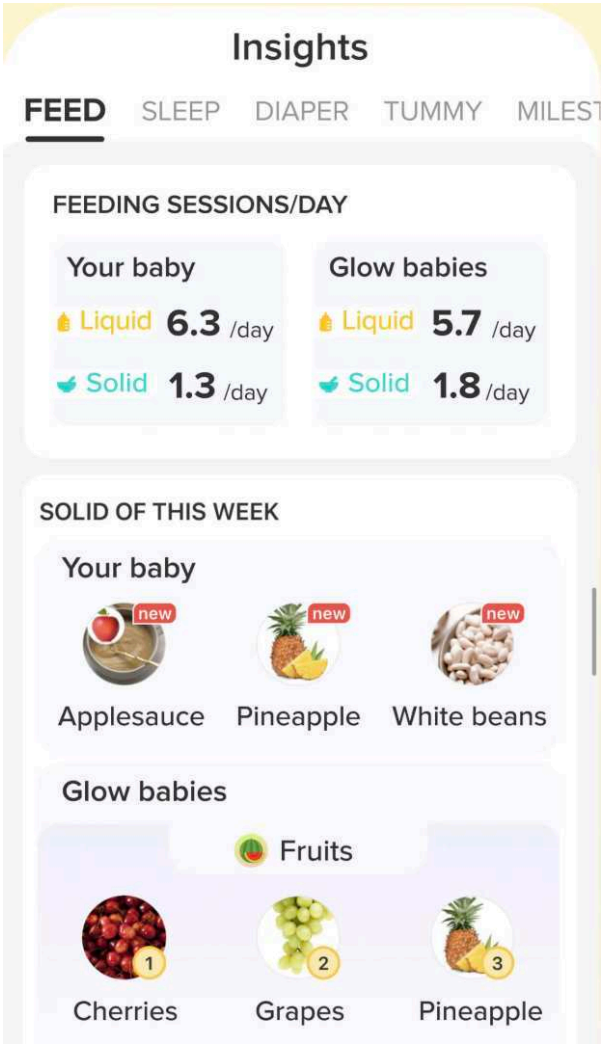


Рисунок 1.3 – Інтерфейс застосунку Glow Baby

1.3 Аналіз функціональності та вимог до системи

1.3.1 Порівняльна характеристика розглянутих рішень

На основі огляду трьох застосунків проведено порівняльний аналіз їхніх ключових переваг і недоліків. Результати наведено в таблиці 1.1.

Таблиця 1.1 – Порівняльна характеристика існуючих рішень

Застосунок	Переваги	Недоліки
Sprout Baby Tracker	1. Широкий журнал активностей 2. Підтримка декількох профілів дітей 3. Надійна робота без мережі	1. Відсутність кривих ВООЗ 2. Синхронізація не в реальному часі 3. Відсутність ШІ-асистента
Huckleberry	1. Глибока аналітика сну 2. Алгоритм SweetSpot 3. Чат з консультантом зі сну	1. Відсутність антропометричного модуля 2. Обмежений безкоштовний функціонал 3. Відсутність ШІ-асистента
Glow Baby	1. Широкий контент та вікові підказки 2. Спільний доступ для батьків 3. База статей про догляд	1. Спрощені графіки без кривих ВООЗ 2. Загальні статті без прив'язки до даних 3. Відсутність ШІ-асистента

Проведений порівняльний аналіз демонструє, що кожна з розглянутих платформ має свої сильні сторони, але водночас і певні обмеження. Sprout Baby Tracker вирізняється зрілим інтерфейсом та широким журналом активностей, але не має підтримки кривих ВООЗ та синхронізації в

реальному часі. Huckleberry пропонує унікальну аналітику сну, проте є вузькоспеціалізованим і не забезпечує комплексного моніторингу. Glow Baby охоплює найширший спектр функцій серед аналогів, однак поступається у точності антропометричних графіків і не має персоналізованого ШІ-асистента. Таким чином, жоден із розглянутих застосунків не поєднує повноцінного антропометричного моніторингу відповідно до стандартів ВООЗ, синхронізації в реальному часі та ШІ-асистента, що обґрунтовує доцільність розроблення нового рішення.

Для наочного позиціонування розроблюваної системи відносно існуючих аналогів складено функціональну матрицю порівняння, наведену в таблиці 1.2.

Таблиця 1.2 – Функціональна матриця порівняння застосунків

Критерій	Sprout Baby Tracker	Huckleberry	Glow Baby	Розроблювана система
Журнал активностей	Так	Частково	Так	Так
Антропометрія з кривими ВООЗ	Ні	Ні	Частково	Так
Синхронізація в реальному часі	Ні	Ні	Ні	Так
ШІ-асистент з педіатричною базою знань	Ні	Ні	Ні	Так
Аналітика режиму сну	Ні	Так	Ні	Частково
Вакцинації та вікові досягнення	Частково	Ні	Частково	Так
Спільний доступ опікунів	Частково	Ні	Так	Так

1.3.2 Вимоги до реалізації застосунку

З урахуванням аналізу існуючих рішень та потреб користувачів сформульовано функціональні та нефункціональні вимоги до розроблюваної системи. Вимоги формувалися з прагненням усунути виявлені недоліки аналогів і забезпечити комплексний інструмент для відстеження розвитку немовляти.

Функціональні вимоги:

- 1) Реєстрація та автентифікація користувача через email і пароль із захищеним зберіганням сесії.
- 2) Підтримка декількох профілів дітей та моделі опікунів: можливість запросити іншого користувача для спільного ведення записів.
- 3) Ведення журналу активностей: годування (грудне молоко, суміш, прикорм), сон, підгузники, купання, температура, ліки.
- 4) Додавання антропометричних вимірювань (маса тіла, зріст, окружність голови) та їх відображення на інтерактивних графіках із перцентильними кривими ВООЗ.
- 5) Управління графіком вакцинацій із відміткою виконаних щеплень.
- 6) Відстеження вікових досягнень дитини з відміткою дати виконання.
- 7) ШІ-асистент для відповідей на педіатричні питання на основі технології RAG з векторним пошуком у перевірній базі знань.
- 8) Синхронізація даних між пристроями в реальному часі через механізм підписки на зміни БД.

Нефункціональні вимоги:

- 1) Кросплатформна підтримка iOS 16.0+ та Android 10+ із нативним відображенням компонентів інтерфейсу.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						13
Зм.	Лист	№ докум.	Підп.	Дата		

- 2) Час відгуку API не більше 500 мс для основних операцій при стабільному мережевому з'єднанні.
- 3) Ізоляція даних між користувачами на рівні СУБД засобами RLS, незалежно від логіки клієнтського застосунку.
- 4) Масштабованість хмарної інфраструктури для обслуговування зростаючої кількості користувачів без зміни архітектури.
- 5) Захист застосунку від несанкціонованої модифікації та запуску на скомпрометованих пристроях.

Визначені функціональні та нефункціональні вимоги є основою для проєктування архітектури системи та вибору технологічного стеку в наступних розділах.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						14
Зм.	Лист	№ докум.	Підп.	Дата		

ВИСНОВОК ДО РОЗДІЛУ 1

У першому розділі проведено аналіз предметної області — системи агрегації та обробки медико-біологічних даних немовляти в мобільному застосунку. Обґрунтовано актуальність розроблення: регулярний педіатричний моніторинг є критично важливим у перші два роки життя дитини, проте більшість батьків не має зручного цифрового інструменту для його ведення з підтримкою реального часу та педіатричним ШІ-асистентом.

Здійснено огляд трьох популярних аналогів — Sprout Baby Tracker, Huckleberry та Glow Baby. Встановлено, що кожен із них орієнтований на певний аспект моніторингу: Sprout Baby Tracker охоплює широкий журнал активностей, Huckleberry спеціалізується на аналізі сну, а Glow Baby пропонує найбільший обсяг контенту серед аналогів. Водночас жоден із розглянутих застосунків не поєднує повноцінного антропометричного моніторингу з кривими ВООЗ, синхронізації між пристроями в реальному часі та ШІ-асистента з перевіреною педіатричною базою знань.

На основі порівняльного аналізу сформульовано вісім функціональних і п'ять нефункціональних вимог до розроблюваної системи. Функціональні вимоги охоплюють автентифікацію, модель опікунів, журнал активностей, антропометричний моніторинг із кривими ВООЗ, вакцинації, досягнення, ШІ-асистент та синхронізацію в реальному часі. Нефункціональні вимоги регламентують продуктивність, безпеку та масштабованість системи.

Визначені вимоги покладено в основу вибору технологічного стеку та проєктування архітектури системи, які розглядаються в наступних розділах.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						15
Зм.	Лист	№ докум.	Підп.	Дата		

РОЗДІЛ 2

ОГЛЯД ТА ВИБІР ТЕХНОЛОГІЙ ДЛЯ ПРОЕКТУВАННЯ СИСТЕМИ

2.1 Загальні принципи побудови кросплатформних мобільних застосунків

2.1.1 Клієнт-серверна архітектура мобільних застосунків

Більшість сучасних мобільних застосунків побудовано за клієнт-серверною архітектурою: клієнтська частина (мобільний застосунок) відповідає за відображення даних і взаємодію з користувачем, тоді як серверна частина забезпечує зберігання, обробку та ізоляцію даних. Такий поділ дозволяє масштабувати компоненти системи незалежно: бекенд може обслуговувати одночасно iOS- та Android-клієнтів без дублювання бізнес-логіки.

У контексті розроблення мобільних клієнтів виокремлюють три основні підходи, що відрізняються балансом між продуктивністю, витратами на розробку та доступом до платформних можливостей.

Нативна розробка передбачає окремі кодові бази для iOS (Swift/Objective-C) та Android (Kotlin/Java). Цей підхід забезпечує найвищу продуктивність, повний доступ до платформних API та найкраще відчуття платформи, однак вимагає підтримки двох незалежних кодових баз, що подвоює витрати на розробку та тестування.

Гібридний підхід базується на вебтехнологіях (HTML/CSS/JavaScript), загорнутих у нативну оболонку за допомогою таких фреймворків, як Ionic або Capacitor. Спільна кодова база суттєво знижує витрати, проте застосунок виконується у WebView, що призводить до помітних затримок

при рендерингу та нестандартної поведінки нативних елементів UI.

Кросплатформна нативна розробка поєднує переваги обох підходів: єдина кодова база транспілюється або компілюється у нативний код для кожної платформи. Серед представників цього підходу виокремлюють React Native від Meta та Flutter від Google. React Native відображає компоненти через нативний міст платформи, тоді як Flutter використовує власний рушій рендерингу Skia/Impeller. Обидва інструменти дозволяють досягти продуктивності, близької до нативної, при значно менших витратах на розробку.

Для розроблюваного застосунку обрано кросплатформну нативну розробку на базі React Native. Ключовими факторами вибору є зрілість екосистеми JavaScript/TypeScript, пряма інтеграція з Supabase SDK та Expo, а також можливість повторного використання вебкомпонентів. На відміну від Flutter, React Native не потребує вивчення мови Dart і органічно вписується в існуючу екосистему npm-пакетів.

2.1.2 Основні компоненти мобільного застосунку

Архітектура розроблюваного застосунку організована у чотири логічних шари з односпрямованою залежністю: вищий шар може звертатись до нижчого, але не навпаки, що запобігає циклічним залежностям і спрощує тестування.

Найближчим до користувача є шар представлення (*Presentation layer*), що містить компоненти UI, екрани та навігаційну структуру. У застосунку він реалізований на базі Expo Router із файловою маршрутизацією: кожен файл у директорії `app/` автоматично відповідає окремому маршруту, а вкладеність директорій визначає ієрархію навігації. Компоненти цього шару отримують дані виключно через пропси або React-хуки і ніколи не звертаються до API напряму.

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		17

Під ним розташований шар бізнес-логіки (*Business logic layer*), відповідальний за трансформацію даних, валідацію та управління станом застосунку. Він реалізований через кастомні React-хуки та глобальне сховище Zustand, у якому зберігаються поточний профіль дитини і стан автентифікаційної сесії. Цей шар виступає посередником між представленням і даними: він формує запити до API та передає готові до відображення об'єкти вгору по стеку.

Шар даних (*Data layer*) забезпечує взаємодію з хмарним API та управління локальним кешем. Він реалізований через клієнт Supabase у поєднанні з TanStack Query, що автоматично кешує відповіді сервера, виконує фонове оновлення при поверненні застосунку на передній план і повторює невдалі запити після відновлення мережевого з'єднання.

Основу стеку складає хмарний бекенд (*Backend layer*) — PostgreSQL з увімкненим RLS, автентифікаційний сервіс Supabase Auth, Realtime-підписки для синхронізації між пристроями та Edge Functions для додаткової серверної логіки. Бекенд не передбачає окремого серверного застосунку: бізнес-правила інкапсульовано безпосередньо у SQL-функції та тригери бази даних, що скорочує кількість рухомих частин системи.

2.2 Огляд технологій для реалізації застосунку

2.2.1 Технології клієнтської частини

React Native — фреймворк від компанії Meta для розроблення нативних мобільних застосунків із використанням JavaScript та TypeScript. На відміну від гібридних рішень, компоненти React Native транслюються безпосередньо у нативні елементи UI кожної платформи без проміжного WebView, що забезпечує продуктивність, порівнянну з нативними застосунками. Єдина кодова база для iOS та Android у поєднанні з великою екосистемою

					ІАЛЦ.467200.003 ПЗ	Аркуш
						18
Зм.	Лист	№ докум.	Підп.	Дата		

npm-пакетів і підтримкою гарячого оновлення (Fast Refresh) суттєво прискорює цикл розробки.

Expo є офіційно рекомендованою надбудовою над React Native, що надає уніфікований SDK для доступу до нативних можливостей пристрою (камера, сповіщення, сенсори) без необхідності писати нативний код на Swift чи Kotlin. На відміну від bare React Native, Expo керує конфігурацією нативних проєктів автоматично через систему плагінів, що спрощує обслуговування і оновлення залежностей. Expo Router реалізує файлову маршрутизацію за аналогією з Next.js: структура директорій app/ безпосередньо визначає навігаційне дерево застосунку, усуваючи потребу в ручному описі маршрутів.

2.2.2 Хмарна платформа та серверна частина

Supabase — відкрита альтернатива Firebase на базі PostgreSQL. Платформа надає такі сервіси:

- реляційна база даних PostgreSQL з повноцінною підтримкою SQL;
- автентифікація (email/пароль, OAuth, magic link);
- REST та GraphQL API, автоматично згенеровані з схеми БД;
- Realtime — двостороння синхронізація змін у таблицях через WebSocket;
- Storage — зберігання файлів і зображень;
- Edge Functions — безсерверні функції на Deno для додаткової бізнес-логіки.

Supabase розгорнуто на хмарній інфраструктурі AWS і масштабується автоматично. Вибір Supabase замість Firebase обумовлений повноцінною підтримкою реляційних запитів, відкритим вихідним кодом та можливістю самостійного розгортання.

2.2.3 Характеристика системи управління базами даних

PostgreSQL — потужна об'єктно-реляційна СУБД з відкритим вихідним кодом. У контексті розроблюваної системи PostgreSQL відіграє центральну роль: зберігає всі медико-біологічні дані, забезпечує транзакційну цілісність і реалізує RLS.

Розширення pgvector додає підтримку векторних типів даних і індексів для пошуку найближчих сусідів у просторі ембедингів. Це дозволяє зберігати векторні представлення педіатричних статей безпосередньо в PostgreSQL і виконувати семантичний пошук у базі знань без зовнішнього векторного сховища.

2.2.4 Управління станом, локалізація та допоміжні бібліотеки

TanStack Query реалізує декларативне управління асинхронним станом: кешування, фонове оновлення, повторні спроби та синхронізацію запитів між компонентами. Бібліотека суттєво спрощує взаємодію клієнта з API та усуває потребу в ручному управлінні станами завантаження й помилок.

Zustand — мінімалістична бібліотека глобального стану для React та React Native. На відміну від Redux, вона не вимагає шаблонного коду і надає простий API для створення сховищ. У розроблюваному застосунку Zustand використовується для зберігання поточного профілю дитини та налаштувань сесії.

i18next — фреймворк інтернаціоналізації для JavaScript, що забезпечує підтримку декількох мов у застосунку. Застосунок підтримує українську та англійську локалізацію з автоматичним визначенням мови пристрою та можливістю ручного перемикавання.

react-native-gifted-charts — бібліотека графіків для React Native, що використовується для відображення антропометричних даних дитини та

стандартних кривих ВООЗ. Бібліотека надає гнучкий API для побудови лінійних і комбінованих графіків безпосередньо на нативному Canvas.

Groq API — платформа для швидкого LLM-inference, що надає доступ до великих мовних моделей (зокрема Llama 3.3 70B) через REST API з надзвичайно малою затримкою завдяки спеціалізованому апаратному прискорювачу LPU.

2.3 Механізми автентифікації та захисту даних

Безпека медико-біологічних даних дитини є критичною вимогою системи. Розроблювана архітектура реалізує захист на двох незалежних рівнях: на рівні транспорту всі запити передаються виключно через HTTPS, а на рівні СУБД доступ до рядків таблиць обмежується механізмом RLS.

Автентифікація здійснюється через Supabase Auth, що видає підписаний JWT-токен після успішного входу. Цей токен передається в заголовок кожного API-запиту та використовується СУБД для визначення поточного користувача.

2.3.1 Row-Level Security та ізоляція даних користувачів

RLS — механізм PostgreSQL, що дозволяє визначати умови видимості рядків таблиці на рівні СУБД. При увімкненому RLS кожен SQL-запит автоматично отримує додаткові умови фільтрації, визначені відповідними політиками безпеки.

У розроблюваній системі кожна таблиця (профілі дітей, активності, вимірювання, вакцинації) має набір RLS-політик, що дозволяють користувачу читати та змінювати лише власні записи. Ідентифікатор користувача витягується безпосередньо з JWT-токена функцією `auth.uid()`, що унеможливлює підробку ідентичності навіть за наявності прямого доступу до API.

Таким чином, RLS надає додатковий рівень захисту, незалежний від логіки застосунку: навіть якщо клієнтський код містить помилку авторизації, СУБД не поверне чужих даних.

2.3.2 Безпека мобільного застосунку

Мобільний застосунок, що обробляє персональні медичні дані, є потенційною ціллю для реверс-інжинірингу та несанкціонованого доступу. Для захисту на рівні клієнтського середовища виконання інтегровано два інструменти.

freeRASP — бібліотека захисту мобільного застосунку в режимі виконання (*Runtime Application Self-Protection*). Вона виявляє загрозливе середовище: root-доступ або jailbreak на пристрої, підключені дебагери, запуск у емуляторі, а також ознаки переупакування або підробки застосунку. У разі виявлення загрози застосунок може заблокувати виконання критичних операцій або завершити сесію.

Sentry — платформа моніторингу помилок і продуктивності. Інтеграція Sentry дозволяє в реальному часі отримувати звіти про необроблені винятки, падіння застосунку та повільні мережеві запити. Це суттєво скорочує час виявлення та виправлення дефектів у виробничому середовищі.

2.4 Стандарти BOO3 та педіатричний моніторинг

BOO3 у 2006 році опублікувала Стандарти зростання дитини (*Child Growth Standards*) — набір перцентильних кривих, що описують нормальний фізичний розвиток дітей від народження до п'яти років. Стандарти побудовані на основі міжнародного мультицентрового дослідження за участю дітей із шести країн і вважаються глобальним еталоном.

Для кожного показника (маса тіла, зріст, окружність голови) та кожної статі визначено п'ять перцентильних ліній: P3, P15, P50, P85 та P97. Значення

					ІАЛЦ.467200.003 ПЗ	Аркуш
						22
Зм.	Лист	№ докум.	Підп.	Дата		

P50 відповідає медіані популяції, P3 — нижній межі норми, P97 — верхній. Відхилення вимірювання за межі P3 або P97 є підставою для консультації з педіатром.

У розроблюваному застосунку перцентильні таблиці ВООЗ вбудовано статично для діапазону 0–24 місяці окремо для хлопчиків і дівчаток. При додаванні нового вимірювання застосунок автоматично визначає перцентиль дитини і відображає її положення на графіку відносно кривих норми. Статичне вбудовування таблиць усуває залежність від мережевого підключення при перегляді антропометричних даних.

2.5 Інтеграція штучного інтелекту на основі RAG

Класичні LLM генерують відповіді виключно на основі знань, закладених під час навчання, і не мають доступу до актуальної або спеціалізованої інформації. Для вирішення цієї проблеми застосовується архітектура RAG.

2.5.1 Поняття векторного пошуку та RAG-архітектури

RAG — підхід, за якого перед генерацією відповіді система виконує семантичний пошук у базі знань і передає знайдені релевантні фрагменти безпосередньо в контекст запиту до мовної моделі. Це дозволяє моделі спиратися на актуальну та перевірену інформацію, а не лише на параметри навчання.

Семантичний пошук реалізується через векторне представлення тексту (*embeddings*): кожен фрагмент документа перетворюється на числовий вектор у багатовимірному просторі. Запит користувача також векторизується, після чого в базі знань шукаються фрагменти з найменшою косинусною відстанню до вектора запиту.

У розроблюваній системі педіатричні статті та рекомендації

зберігаються в PostgreSQL із розширенням pgvector. При надходженні запиту від користувача:

- 1) запит векторизується моделлю ембедингів;
- 2) у таблиці knowledge_chunks виконується пошук k найближчих фрагментів;
- 3) знайдені фрагменти разом із запитом передаються до Groq API;
- 4) мовна модель генерує відповідь, спираючись на наданий контекст.

Така архітектура забезпечує точні та верифіковані відповіді на педіатричні питання, мінімізуючи ризик галюцинацій (*hallucinations*) з боку мовної моделі.

ВИСНОВОК ДО РОЗДІЛУ 2

У другому розділі визначено технологічний стек та архітектурні принципи розроблюваної системи. Обґрунтовано вибір клієнт-серверної архітектури з чотирма логічними шарами: представлення, бізнес-логіки, даних і хмарного бекенду. Показано, що кросплатформна нативна розробка є оптимальним підходом для охоплення iOS та Android при єдиній кодовій базі.

Обрано React Native та Expo як основу клієнтської частини. React Native забезпечує нативне відображення компонентів без WebView, а Expo спрощує доступ до платформних можливостей і управління конфігурацією нативних проєктів через систему плагінів.

Хмарну платформу Supabase на базі PostgreSQL обрано завдяки підтримці реляційних запитів, механізму RLS для ізоляції даних між користувачами та вбудованому сервісу синхронізації в реальному часі. Розширення pgvector дозволяє зберігати векторні ембединги педіатричної бази знань безпосередньо в СУБД, без потреби у зовнішньому векторному сховищі.

Для управління станом обрано TanStack Query — для асинхронних запитів з автоматичним кешуванням, та Zustand — для глобального стану застосунку. Локалізацію забезпечує i18next з підтримкою української та англійської мов, а react-native-gifted-charts використовується для побудови антропометричних графіків із кривими BOO3. Захист виконуваного середовища реалізовано через freeRASP, моніторинг помилок — через Sentry.

Розглянуто стандарти BOO3 щодо фізичного розвитку дітей і способ їх статичної інтеграції у застосунок у вигляді перцентильних таблиць для діапазону 0–24 місяці. Описано архітектуру III-асистента на базі

					ІАЛЦ.467200.003 ПЗ	Аркуш
						25
Зм.	Лист	№ докум.	Підп.	Дата		

RAG: семантичний пошук релевантних фрагментів педіатричних знань у PostgreSQL через pgvector забезпечує верифіковані відповіді з мінімальним ризиком галюцинацій.

РОЗДІЛ 3

РЕАЛІЗАЦІЯ СИСТЕМИ

3.1 Архітектура системи

3.1.1 Загальна архітектурна схема застосунку

Розроблювана система є кросплатформним мобільним застосунком із хмарним бекендом. Загальну архітектурну схему системи наведено у додатку Д1. Система складається з двох основних частин: клієнтського мобільного застосунку, що функціонує на платформах iOS та Android, та хмарного бекенду на базі Supabase. Такий розподіл відповідає архітектурному патерну «тонкий клієнт», де уся логіка збереження та читання даних, перевірка прав доступу і бізнес-правила зосереджені на стороні сервера, а клієнт лише відображає дані та надсилає запити.

Клієнтський застосунок реалізовано засобами React Native та Expo. Він не містить власної серверної логіки та взаємодіє з хмарним бекендом виключно через REST API та WebSocket-підписки. Такий підхід дозволяє розгортати оновлення бекенду незалежно від оновлень застосунку в магазинах додатків, що суттєво скорочує час між виправленням серверної помилки та доступністю виправлення для кінцевих користувачів. Крім того, відсутність серверної логіки на клієнті зменшує поверхню атаки: навіть за умови компрометації клієнтського коду злоумисник не отримає доступу до даних іншого користувача.

Хмарний бекенд побудовано на платформі Supabase і містить чотири функціональних компоненти: PostgreSQL як основне сховище даних, Supabase Auth для управління автентифікацією та сесіями, Supabase Storage для зберігання медіафайлів, а також Edge Functions для виконання

серверної бізнес-логіки, зокрема обробки запитів до ШІ-асистента. Кожен із компонентів розгортається і обслуговується платформою Supabase, що звільняє розробника від необхідності адміністрування серверної інфраструктури.

Взаємодія між клієнтом і бекендом здійснюється через `supabase-js` — офіційний JavaScript-клієнт Supabase, що інкапсулює формування HTTP-запитів, управління JWT-токенами та підтримку Realtime-підписок. Усі запити передаються по захищеному каналу HTTPS; для WebSocket-з'єднань використовується протокол WSS. Завдяки типізованому клієнту з автоматично згенерованими TypeScript-інтерфейсами схеми бази даних будь-яка невідповідність між клієнтськими запитами та фактичною схемою виявляється на етапі компіляції, а не в середовищі продакшну.

При першому запуску застосунок перевіряє наявність збереженої сесії в `AsyncStorage`. Якщо сесія дійсна, застосунок переходить до головного екрану; в іншому випадку відображається екран автентифікації. Після успішного входу Supabase Auth видає пару токенів: JWT доступу (термін дії 1 година) та токен оновлення. Автоматична ротація токенів відбувається при переведенні застосунку на передній план, що унеможливорює тривале використання скомпрометованого токена доступу.

3.1.2 Архітектурні особливості клієнтської частини

Клієнтський застосунок організовано за принципом `feature-based` архітектури: увесь код згруповано по функціональних доменах, а не за технічними ролями. Така структура забезпечує зв'язність коду усередині кожного модуля і мінімізує залежності між модулями, що значно спрощує навігацію по кодовій базі та локалізацію помилок. На відміну від традиційного підходу, де компоненти, хуки та утиліти одного домену

розкидані по різних директоріях, feature-based підхід концентрує весь код певної функціональності в одному місці.

Директорія `app/` містить навігаційні екрани у форматі Expo Router, де структура директорій безпосередньо відображається у дерево маршрутів застосунку. Директорія `features/` зберігає доменну логіку дев'яти функціональних областей: `feedings`, `sleeps`, `diapers`, `measurements`, `milestones`, `vaccinations`, `children`, `auth` та `assistant`. Директорія `components/` містить перевикористовувані компоненти UI, що не належать до жодного конкретного домену. Директорії `stores/`, `lib/`, `constants/` та `i18n/` зберігають глобальні стани, клієнтські інтеграції, дизайн-токени та файли локалізації відповідно. Така організація дозволяє розробнику, що не знайомий з проектом, за назвою директорії відразу зрозуміти, яку функціональність вона реалізує, що знижує поріг входу для нових учасників команди.

Кожна директорія в `features/` містить файл `queries.ts` з React-хуками для отримання і мутації даних через TanStack Query та Supabase. Хуки отримання даних побудовані на `useQuery` і повертають стандартизовані об'єкти з полями `data`, `isLoading`, `isError` та `refetch`, що забезпечує однорідний інтерфейс взаємодії з даними по всьому застосунку. Хуки мутацій побудовані на `useMutation` і автоматично інвалідують відповідні ключі кешу після успішного виконання операції, завдяки чому UI оновлюється без явного виклику `refetch` з боку компонентів.

Шар представлення відокремлений від шару даних: екрани у `app/` отримують дані виключно через хуки з `features/`, а не звертаються до Supabase напряму. Це дозволяє тестувати бізнес-логіку незалежно від компонентів UI і спрощує заміну джерела даних без переписування екранів. Наприклад, якщо у майбутньому виникне необхідність замінити Supabase на інший бекенд, зміни знадобляться лише у файлах `queries.ts` кожного

модуля, тоді як усі екрани і компоненти залишаються незмінними.

3.1.3 Архітектура хмарної серверної частини

Хмарний бекенд системи побудовано на базі платформи Supabase, що надає повний набір інфраструктурних сервісів без необхідності розробки та підтримки власного серверного застосунку. Такий підхід дозволяє зосередити весь обсяг розробки на клієнтській логіці та предметній області застосунку, не витрачаючи ресурси на розробку і підтримку типових серверних компонентів, таких як автентифікація, авторизація, файлове сховище та синхронізація в реальному часі. Усі компоненти бекенду автоматично масштабуються залежно від навантаження без участі розробника.

Центральним компонентом бекенду є PostgreSQL — реляційна СУБД, яка зберігає всі предметні дані системи та реалізує ізоляцію даних між користувачами через механізм RLS (Row-Level Security), детально описаний у підрозділі 3.2.2. Поверх PostgreSQL розгорнуто PostgREST — проміжний шар, що автоматично генерує REST API для кожної таблиці і представлення бази даних. Клієнтський код звертається до бази даних через `supabase-js`, який транслює типізовані запити у відповідні HTTP-запити до PostgREST, а той виконує відповідний SQL-запит від імені аутентифікованого користувача.

Supabase Auth реалізує управління обліковими записами і сесіями. Сервіс підтримує реєстрацію і вхід через email та пароль, а також авторизацію через Google за протоколом OAuth 2.0. OAuth (Open Authorization) — відкритий стандарт делегованої авторизації, що дозволяє користувачу надати застосунку обмежений доступ до свого акаунту у стороннього провайдера (Google, Apple тощо) без передачі пароля: замість цього провайдер видає одноразовий код, який застосунок обмінює на токени доступу. При успішній автентифікації Supabase Auth генерує

					ІАЛЦ.467200.003 ПЗ	Аркуш
						30
Зм.	Лист	№ докум.	Підп.	Дата		

підписані JWT-токени, що містять ідентифікатор користувача у полі `sub` та метадані сесії. Ці токени передаються у заголовку `Authorization: Bearer` кожного запиту до PostgREST, де PostgreSQL витягує ідентифікатор користувача функцією `auth.uid()` для застосування RLS-фільтрів. Завдяки такій інтеграції авторизаційна перевірка відбувається безпосередньо при виконанні SQL-запиту, а не у додатковому прошарку між клієнтом і базою даних.

Supabase Realtime транслює зміни у таблицях бази даних клієнтам через постійне WebSocket-з'єднання на основі механізму логічної реплікації PostgreSQL, забезпечуючи миттєву синхронізацію між пристроями без регулярного опитування сервера. Детальна реалізація підписок описана у підрозділі 3.3.4.

Edge Functions — це ізольовані безсерверні функції, що виконуються у хмарній інфраструктурі Supabase на базі середовища виконання Deno. Deno — це сучасна альтернатива Node.js, яка виконує TypeScript-код без попередньої компіляції, має вбудовану підтримку Web API і використовує явну модель дозволів, що підвищує безпеку ізольованого середовища. Edge Functions використовуються для бізнес-логіки, яка потребує захищеного доступу до сторонніх API: секретні ключі зберігаються виключно на сервері у Supabase Secrets і ніколи не потрапляють до клієнтського коду.

3.1.4 Архітектура бази даних

База даних системи реалізована у PostgreSQL і містить вісім предметних таблиць. Детальну ER-діаграму бази даних наведено у додатку Д2. Всі таблиці використовують ідентифікатори типу `uuid`, що генеруються функцією `gen_random_uuid()`, — на відміну від автоінкрементних цілочисельних ідентифікаторів, UUID не дозволяють атакуючому передбачити ідентифікатор запису іншого користувача та

виконати цільований запит на його отримання.

Центральною сутністю є таблиця `children`, що зберігає профілі дітей: ідентифікатор, повне ім'я, дата народження, стать, нотатки та посилання на аватар у Supabase Storage. Зв'язок між профілями дітей та користувачами системи реалізовано через окрему таблицю `caregivers`, що містить пару (`profile_id`, `child_id`) та роль доглядача (`owner` або `viewer`). Такий підхід реалізує відношення «багато-до-багатьох» між користувачами і дітьми та дозволяє одній дитині мати кількох доглядачів з різними рівнями доступу, що відповідає реальному сімейному сценарію використання застосунку.

Таблиці активностей `feedings`, `sleeps` та `diapers` зберігають відповідні події для конкретної дитини з тимчасовими мітками початку і завершення. Таблиця `feedings` містить поле `kind` з перерахуванням допустимих типів годування (`breast_left`, `breast_right`, `bottle_breast`, `bottle_formula`, `solid`), а також поля тривалості сеансу та кількості молока в мілілітрах. Для тривалих активностей — годування та сон — реалізовано підтримку активних сеансів: запис зі значенням `NULL` у полі `ended_at` вважається незавершеним поточним сеансом, що дозволяє відображати активний таймер на головному екрані без зберігання проміжного стану у клієнтському застосунку.

Таблиця `growth_measurements` зберігає антропометричні вимірювання дитини з типом показника (`weight`, `height`, `head_circumference`), числовим значенням та одиницею вимірювання. Таблиці `milestone_achievements` та `vaccination_records` фіксують факти досягнення вікових міток та вакцинацій з посиланням на шаблони (`milestone_templates` та `vaccination_templates`), що зберігаються в окремих системних таблицях, заповнених стандартними педіатричними даними. Розподіл на таблиці шаблонів та таблиці досягнень дозволяє централізовано оновлювати набір контрольних точок розвитку без

необхідності міграції індивідуальних записів кожного користувача.

3.2 Реалізація серверної частини

3.2.1 Проєктування та реалізація бази даних

Схема бази даних розроблена з урахуванням вимог нормалізації до третьої нормальної форми та особливостей механізму RLS у Supabase. Дотримання нормалізації усуває надлишковість даних: наприклад, інформація про шаблони вакцинацій зберігається один раз у таблиці `vaccination_templates`, а таблиця `vaccination_records` містить лише зовнішній ключ на відповідний шаблон. Усі предметні таблиці містять первинний ключ типу `uuid` з автоматичною генерацією через `gen_random_uuid()`, що унеможливорює передбачення ідентифікаторів атакуючим та спрощує горизонтальне масштабування бази даних у майбутньому.

Таблиця `children` є головною сутністю схеми і містить такі поля: `id uuid PRIMARY KEY DEFAULT gen_random_uuid()`, `full_name text NOT NULL`, `date_of_birth date NOT NULL`, `sex text CHECK (sex IN ('male', 'female', 'unspecified'))`, `notes text`, `avatar_url text` та `created_at timestamptz DEFAULT now()`. Обмеження `CHECK` на поле `sex` забезпечує цілісність даних на рівні СУБД і не дозволяє клієнту зберегти довільне рядкове значення для цього поля. Таблиця `caregivers` реалізує зв'язок між користувачами та дітьми через поля `profile_id uuid REFERENCES auth.users NOT NULL` та `child_id uuid REFERENCES children NOT NULL` з унікальним обмеженням на цю пару, що запобігає дублюванню записів про одного й того самого доглядача.

Таблиця `feedings` містить такі ключові поля: `child_id uuid REFERENCES children NOT NULL`, `kind text NOT NULL`, `started_at`

timestampz NOT NULL, ended_at timestampz (зі значенням NULL для активного сеансу), amount_ml integer для кількості молока у пляшковому годуванні та created_by uuid REFERENCES auth.users для фіксації автора запису. Таблиці sleeps і diapers побудовано за аналогічним принципом. У таблиці growth_measurements додано поле value numeric(6,2) NOT NULL для числового значення та unit text для одиниці вимірювання, що забезпечує точність до двох десяткових знаків та підтримує зберігання як метричних, так і імперіальних одиниць.

При створенні нової дитини база даних автоматично додає запис у таблицю caregivers завдяки тригеру on_child_created, що виконується після кожної вставки у таблицю children. Тригер визначає поточного аутентифікованого користувача через auth.uid() та додає пару (auth.uid(), new.id) з роллю owner. Такий підхід гарантує, що кожна дитина завжди має хоча б одного власника, і спрощує клієнтський код: для реєстрації нової дитини потрібен лише один INSERT-запит до таблиці children, а зв'язок із поточним користувачем створюється автоматично на стороні СУБД без додаткових запитів з клієнта.

3.2.2 Реалізація Row-Level Security

RLS є ключовим механізмом ізоляції даних у системі. Після ввімкнення RLS для таблиці PostgreSQL автоматично застосовує визначені політики до кожного запиту незалежно від того, яким способом цей запит надійшов — через API, прямий SQL-клієнт або будь-який інший інтерфейс. Це означає, що захист даних реалізований не на рівні логіки застосунку, а на рівні СУБД, і не може бути обійдений навіть при помилках у клієнтському коді. Для таблиці children визначено дві політики. Перша дозволяє читання лише тим користувачам, для яких існує відповідний запис у таблиці caregivers:

```

CREATE POLICY "caregivers can view children"
ON children FOR SELECT
USING (
    EXISTS (
        SELECT 1 FROM caregivers
        WHERE caregivers.child_id = children.id
        AND caregivers.profile_id = auth.uid()
    )
);

```

Друга політика дозволяє вставку будь-якому аутентифікованому користувачеві, оскільки тригер `on_child_created` автоматично призначить його власником щойно створеного профілю дитини:

```

CREATE POLICY "authenticated users can create children"
ON children FOR INSERT
WITH CHECK (auth.uid() IS NOT NULL);

```

Для таблиць активностей (`feedings`, `sleeps`, `diapers`, `growth_measurements`) RLS-політики перевіряють доступ через таблицю `caregivers` аналогічно до таблиці `children`, але умова перевіряє `child_id` запису активності. Наприклад, для `feedings` SELECT-політика виглядає так:

```

CREATE POLICY "caregivers can view feedings"
ON feedings FOR SELECT
USING (
    EXISTS (
        SELECT 1 FROM caregivers
        WHERE caregivers.child_id = feedings.child_id
        AND caregivers.profile_id = auth.uid()
    )
);

```

)
);

Аналогічні політики для операцій INSERT, UPDATE та DELETE гарантують, що користувач може змінювати лише дані тих дітей, доглядачем яких він є. Функція `auth.uid()` повертає ідентифікатор поточного користувача з JWT-токена, переданого в заголовок запиту; якщо токен відсутній або недійсний, функція повертає NULL, і жодна RLS-політика не виконується, тобто запит від неаутентифікованого користувача автоматично повертає порожній результат для всіх захищених таблиць.

3.2.3 Реалізація Edge Function та III-асистента

III-асистент реалізований у вигляді Edge Function з назвою `ask-assistant`. Edge Function — це окремий TypeScript-модуль, що розгортається командою Supabase CLI і виконується у пісочниці Deno безпосередньо на серверах Supabase. При виклику функція отримує HTTP-запит від клієнта, автентифікує його через JWT-токен, виконує необхідні запити до зовнішніх сервісів (OpenAI, Groq) і повертає відповідь. Такий підхід забезпечує захист секретних ключів зовнішніх API: ключі OpenAI та Groq зберігаються виключно у захищеному сховищі Supabase Secrets і ніколи не потрапляють у клієнтський код або публічний репозиторій. Клієнтський застосунок передає до функції два параметри: текст запиту користувача (`prompt`) та вік дитини у місяцях (`childAgeMonths`).

Функція реалізує RAG-архітектуру, що складається з трьох послідовних кроків. По-перше, вхідний текст запиту перетворюється на вектор ембедингу розмірністю 1536 за допомогою моделі `text-embedding-3-small` від OpenAI. По-друге, отриманий вектор порівнюється з векторами фрагментів педіатричної бази знань у таблиці `knowledge_chunks` через оператор

косинусної схожості <=> розширення pgvector:

```
SELECT content, source_title
FROM knowledge_chunks
ORDER BY embedding <=> query_embedding
LIMIT 5;
```

По-третє, п’ять найближчих фрагментів разом із запитом і системним промптом, що враховує вік дитини, передаються до Groq API для генерації відповіді моделлю Llama 3.3 70B. Системний промпт обмежує модель відповідями у сфері педіатрії та зобов’язує її посилатися на надані фрагменти як на джерела. Функція повертає об’єкт з полями answer (відповідь моделі) та sources (список використаних джерел).

База знань knowledge_chunks сформована з фрагментів педіатричних настанов ВООЗ та Американської академії педіатрії. Кожен фрагмент являє собою текстовий абзац довжиною 200–400 tokenів із векторним представленням, індексованим за допомогою методу IVFFlat розширення pgvector. Системний промпт функції складається з трьох блоків: роль асистента (педіатричний помічник у сфері дитячого здоров’я), контекст дитини (вік у місяцях) та інструкція відповідати лише на основі наданих фрагментів. Такий підхід мінімізує галюцинації моделі та забезпечує прозорість для батьків, які можуть самостійно перевірити зазначені джерела. Екран взаємодії з ШІ-асистентом наведено на рисунку 3.1.

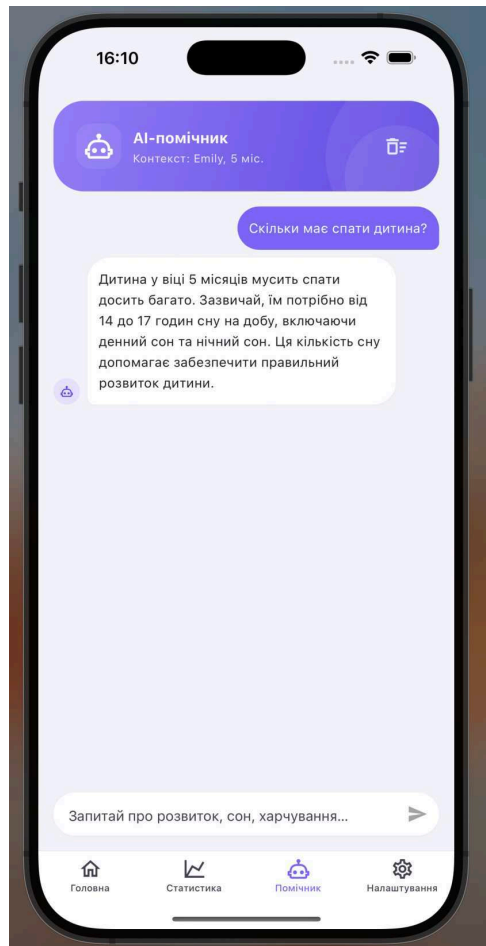


Рисунок 3.1 – Екран ШІ-асистента з відповіддю на запит про сон дитини

3.3 Реалізація клієнтської частини

3.3.1 Організація структури модулів

Реалізація клієнтської частини дотримується єдиного шаблону в усіх функціональних модулях: кожна директорія у `features/` реалізує повний вертикальний зріз одного домену — від типів та запитів до компонентів відображення. Шар даних кожного модуля будується на основі `TanStack Query` і складається з файлу `queries.ts` із хуками отримання даних (`useQuery`) та мутацій (`useMutation`), а також окремих файлів для типів і утиліт домену. Ієрархічна система ключів кешу забезпечує автоматичну інвалідацію при мутаціях без ручного виклику перезавантаження даних з боку компонентів.

Наприклад, модуль годувань містить у файлі `features/feedings/queries.ts` шість хуків: `useFeedingsForDay` та `useFeedingsInRange` для отримання записів за різними часовими діапазонами, `useActiveFeeding` для відстеження незавершеного сеансу, а також `useStartFeeding`, `useStopFeeding` та `useCreateFeeding` для операцій запису. Ключі кешу побудовано ієрархічно: `['feedings', childId]` для всіх записів дитини та `['feedings', childId, 'day', date]` для записів конкретного дня. Завдяки цій ієрархії мутація `useCreateFeeding` може одночасно інвалідувати кеш поточного дня і загальний кеш дитини одним викликом з шаблоном `['feedings', childId]`, що охоплює всі підпорядковані ключі.

Спільні компоненти UI зосереджені в директорії `components/` і поділені на логічні групи залежно від призначення. Контейнерні компоненти `ScreenContainer` та `FormScreen` реалізують стандартні макети екранів з урахуванням безпечних зон пристрою (`SafeAreaView`) та підтримки прокручування. Компоненти форм `AppTextInput`, `DateField` та `KindGrid` забезпечують єдиний візуальний стиль полів введення даних по всьому застосунку. Графічні компоненти `DailyBarChartCard` та `MeasurementChartCard` інкапсулюють логіку побудови та відображення графіків. Усі компоненти параметризовані через TypeScript-інтерфейси та підтримують темну і світлу теми через хук `useTheme` бібліотеки `React Native Paper`. Для реалізації плавних анімацій у спільних компонентах використовується бібліотека `react-native-reanimated`, що виконує анімації безпосередньо у нативному потоці без передачі даних через JavaScript bridge на кожному кадрі. Завдяки цьому анімації зберігають частоту 60 кадрів на секунду навіть під час інтенсивного оновлення UI. `Reanimated` застосовується у `BottomSheet` — модальному аркуші, що виїжджає знизу екрана, де `withSpring` та `withTiming` анімують переміщення

панелі по вертикалі та прозорість фону, — та у `ActiveChildPanel`, де `withSpring` анімує індикатор активної дитини при переключенні між профілями. Спільні значення (`useSharedValue`) зберігаються у нативному потоці, а стилі обчислюються через `useAnimatedStyle` без участі JS-рантайму під час відтворення анімації.

Локалізація реалізована бібліотекою `i18next` у поєднанні з `react-i18next` із файлами перекладів у директорії `i18n/`. Застосунок підтримує дві мови: українську (за замовчуванням) та англійську. Обрана мова зберігається у `AsyncStorage` та застосовується при наступному запуску без необхідності повторного вибору. Всі рядки у компонентах UI отримуються через хук `useTranslation`, що забезпечує централізоване управління текстовим контентом і спрощує додавання нових мов у майбутньому.

Статична типізація всього проєкту забезпечується TypeScript у суворому режимі (`"strict": true` у `tsconfig.json`), що вмикає перевірку на `null/undefined`, заборону неявних типів `any` та суворий контроль типів параметрів функцій. Псевдонім шляху `@/*` дозволяє імпортувати будь-який модуль відносно кореня проєкту без відносних шляхів виду `../.../..`, що скорочує кількість помилок при переміщенні файлів. Статичний аналіз коду виконується ESLint із пресетом `eslint-config-expo`, який охоплює правила для React, React Native, TypeScript та доступності UI-компонентів. Разом TypeScript і ESLint утворюють двошаровий бар'єр якості: компілятор відловлює структурні помилки типів, а лінтер — проблеми стилю, небезпечні патерни та порушення правил хуків React.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						40
Зм.	Лист	№ докум.	Підп.	Дата		

3.3.2 Моніторинг помилок та безпека середовища виконання

Для відстеження збоїв і аномалій у продакшн-середовищі інтегровано бібліотеку `@sentry/react-native`. Sentry — це платформа моніторингу помилок, що автоматично перехоплює необроблені винятки, збої JavaScript-рантайму та відмови нативних модулів, після чого передає структуровані звіти на хмарний дашборд із повним стек-трейсом, інформацією про пристрій, версію застосунку та контекст користувача. Ініціалізація виконується одноразово при запуску застосунку:

```
Sentry.init({  
  dsn: process.env.EXPO_PUBLIC_SENTRY_DSN,  
  tracesSampleRate: 0.2,  
});
```

Параметр `tracesSampleRate: 0.2` означає, що трасування продуктивності збирається для 20% сесій, що дозволяє виявляти повільні мережеві запити та вузькі місця рендерингу без суттєвого навантаження на інфраструктуру. DSN-адреса (ідентифікатор проєкту Sentry) передається через змінну оточення `EXPO_PUBLIC_SENTRY_DSN` і не зберігається в репозиторії.

Для захисту від запуску застосунку у скомпрометованому середовищі інтегровано бібліотеку `freerasp-react-native`. Вона виконує перевірки цілісності середовища виконання: виявляє jailbreak (iOS) та root-доступ (Android), присутність hooking-фреймворків (Frida, Xposed), запуск у емуляторі, а також модифікацію бінарного файлу застосунку. При виявленні загрози бібліотека передає подію до Sentry для фіксації у системі моніторингу та виконує налаштовану реакцію. Така інтеграція утворює дворівневий захист: Sentry фіксує помилки у штатному режимі, а freeRASP реагує на цілеспрямовані спроби маніпуляцій із застосунком.

3.3.3 Навігаційна структура та управління станом

Навігаційна структура застосунку реалізована засобами `Expo Router`, що відображає файлову систему директорії `app/` безпосередньо у дерево маршрутів. Такий підхід усуває необхідність ручної реєстрації маршрутів: додавання нового файлу `app/feedings/new.tsx` автоматично робить маршрут `/feedings/new` доступним у навігаційному стеку. Детальну структурну схему навігації наведено у додатку Д3. `Expo Router` забезпечує також типобезпечну навігацію: при виклику `router.push('/feedings/new')` TypeScript перевіряє наявність відповідного файлу у директорії `app/`, що виключає помилки навігації на етапі компіляції.

Кореневий файл `app/_layout.tsx` є точкою входу навігаційного стеку. Він обгортає все дерево компонентів у провайдери: `AuthProvider` для управління сесією, `QueryClientProvider` для `TanStack Query` та `PaperProvider` для теми матеріального дизайну. На цьому ж рівні ініціалізуються `Realtime`-підписки та компонент `AuthGate`, що реалізує логіку охорони маршрутів. Основний стек містить два вкладені навігатори: вкладкову навігацію з чотирма головними екранами («Головна», «Статистика», «Помічник», «Налаштування») та модальний стек із формами введення даних, що відображаються поверх вкладок як стандартні модальні вікна платформи.

Глобальний стан застосунку зберігається у двох `Zustand`-сховищах з персистентністю через `AsyncStorage`. Сховище `activeChild` містить ідентифікатор поточно обраної дитини; при зміні цього ідентифікатора всі `React Query` хуки, що залежать від `childId`, автоматично виконують повторний запит до API. Такий реактивний ланцюжок забезпечує миттєве оновлення всього інтерфейсу при переключенні між профілями дітей без

явного перезавантаження будь-якого екрана. Сховище `themePreference` зберігає обраний режим теми (`system`, `light` або `dark`) і передається в `PaperProvider` для застосування відповідної теми матеріального дизайну.

3.3.4 Зберігання файлів та синхронізація в реальному часі

`Supabase Storage` використовується для зберігання аватарів дітей. При завантаженні зображення застосунок обирає фото з галереї пристрою через `expo-image-picker` та стискає його до максимального розміру 512×512 пікселів для зменшення обсягу переданих даних. Стиснуте зображення завантажується у бакет `avatars` за детермінованим шляхом `{child_id}/avatar.jpg`, після чого публічний URL зображення зберігається у полі `avatar_url` таблиці `children`. Використання детермінованого шляху замість генерованого імені файлу дозволяє автоматично перезаписувати попередній аватар при завантаженні нового без необхідності видалення старого файлу та оновлення посилання.

Синхронізація між пристроями реалізована через `Supabase Realtime` у файлі `lib/realtime.ts`. При активації основного стеку застосунок підписується на канал `child:{activeChildId}`, що відслідковує зміни у таблицях активностей конкретної дитини. Фільтр на рівні каналу обмежує трафік: клієнт отримує лише події, де `child_id` збігається з ідентифікатором активної дитини, а не всі зміни у таблиці. При отриманні події `postgres_changes` `React Query` клієнт інвалідує відповідний ключ кешу, що ініціює фонове оновлення даних:

`supabase`

```
.channel(`child:${childId}`)
.on('postgres_changes',
  { event: '*', schema: 'public', table: 'feedings',
```

```

        filter: `child_id=eq.${childId}` },
    () => queryClient.invalidateQueries(
        { queryKey: ['feedings', childId] })
    )
    .subscribe();

```

При зміні активної дитини у сховищі `activeChild` попередня підписка скасовується, а нова підписка встановлюється для нового ідентифікатора. Такий підхід запобігає накопиченню неактуальних підписок та зайвому мережевому трафіку. Завдяки реалізованому механізму синхронізації, якщо один із батьків додає запис про годування зі свого пристрою, застосунок іншого батька оновлює відображення автоматично протягом 1–2 секунд без необхідності ручного оновлення сторінки.

3.3.5 Реалізація клієнтської автентифікації

Автентифікація реалізована через `AuthProvider` — React-контекст, що обгортає весь застосунок і надає сесію та методи автентифікації всім дочірнім компонентам. При монтуванні провайдер викликає `supabase.auth.getSession()` для отримання збереженої сесії з `AsyncStorage` пристрою. Якщо сесія знайдена, провайдер перевіряє час її закінчення: якщо до закінчення залишається менше 60 хвилин, виконується автоматичне оновлення токена через `supabase.auth.refreshSession()`. У разі невдачі оновлення (наприклад, коли токен відкликано з боку сервера) сесія вважається недійсною і користувача перенаправляють на екран входу.

Для реєстрації та входу через email і пароль реалізовано відповідні мутації у файлі `features/auth/mutations.ts`. Помилки автентифікації, що повертаються Supabase Auth, перехоплюються і відображаються у формі у вигляді зрозумілих повідомлень для користувача: «Невірний email або

пароль», «Email вже зареєстровано» тощо. Вхід через Google реалізовано за допомогою OAuth 2.0 потоку з бібліотекою expo-web-browser: застосунок відкриває системний браузер із сторінкою авторизації Google, після успішного проходження якої браузер перенаправляє на deep-link застосунку з одноразовим кодом авторизації, який supabase-js автоматично обмінює на пару токенів доступу.

Компонент AuthGate у кореневому `_layout.tsx` підписується на зміни стану автентифікації через `supabase.auth.onAuthStateChange` та умовно відображає або основний стек із вкладками, або стек автентифікації. Завдяки реактивній підписці стан навігації оновлюється миттєво при зміні сесії — як при вході, так і при виході. Екран автентифікації побудований за принципом єдиної точки входу: один екран обслуговує як вхід, так і перехід до реєстрації. Верхня частина екрана є брендовою зоною з градієнтним фіолетовим фоном та логотипом застосунку, нижня — білою карткою з двома варіантами автентифікації: формою email/пароль та кнопкою «Продовжити з Google». Перемикач мови у правому верхньому куті дозволяє змінювати локалізацію безпосередньо на екрані входу без необхідності авторизації. Після успішної реєстрації застосунок автоматично виконує вхід і перенаправляє на екран створення першого профілю дитини. Екран автентифікації наведено на рисунку 3.2.

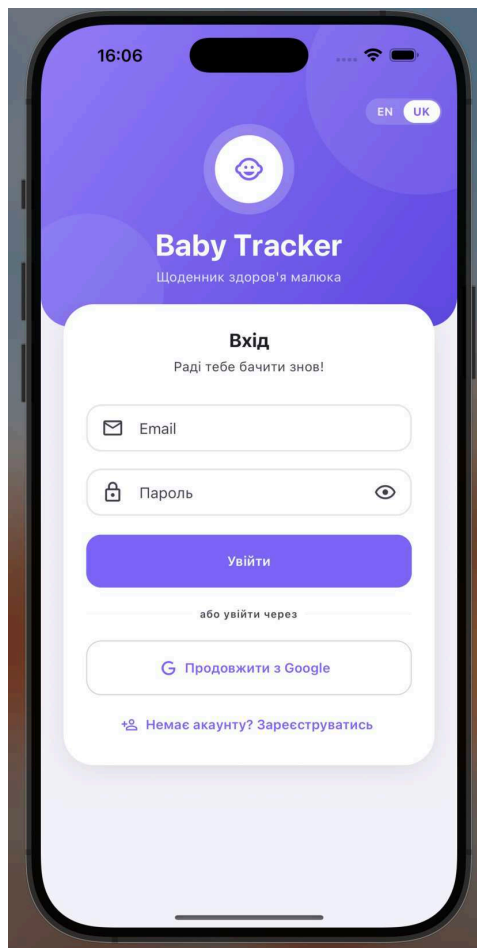


Рисунок 3.2 – Екран автентифікації застосунку

3.3.6 Реалізація ключових екранів та компонентів

Головний екран застосунку (`app/(tabs)/index.tsx`) є центральним інформаційним хабом і першим, що бачить користувач після входу. У верхній частині відображається компонент `HeroCard` — вітальна картка з іменем поточного користувача та датою; нижче — картка активної дитини з аватаром, ім'ям та кількістю повних місяців від дати народження. Рядок лічильників `StatsRow` відображає три агреговані показники поточного дня: кількість годувань, загальний час сну та кількість змін підгузків. Якщо є активний незавершений сеанс, відображається також компонент `ActiveTimerCard` з таймером у реальному часі та кнопкою зупинки. Сітка швидких дій надає прямий доступ до форм введення всіх типів активностей, а хронологічна стрічка подій у нижній частині агрегує записи з усіх доменів

у зворотному хронологічному порядку. Навігація по дням реалізована через кнопки «назад» та «вперед», що змінюють значення `selectedDay` через `subDays/addDays` з `date-fns`, дозволяючи переглядати стрічку будь-якого минулого дня без переходу на окремий екран. Головний екран застосунку наведено на рисунку 3.3.

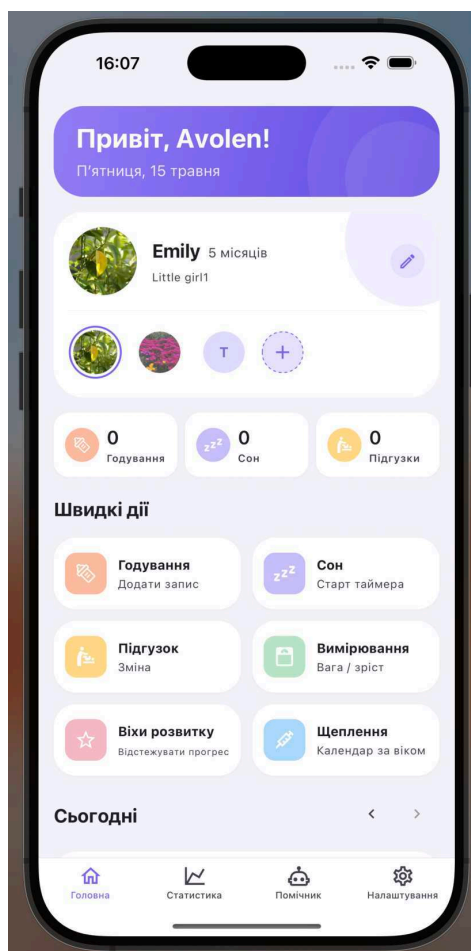


Рисунок 3.3 – Головний екран застосунку

Екран статистики (`app/(tabs)/stats.tsx`) відображає аналітику за останні 7 днів у вигляді трьох стовпчастих графіків: кількість годувань, загальний час сну та кількість змін підгузків по днях тижня. Компонент `DailyBarChartCard` для кожного показника отримує масив значень за 7 днів і будує графік засобами бібліотеки `react-native-gifted-charts`, де кожен стовпець відповідає одному дню. Для аналізу режиму сну реалізовано окремий компонент `SleepPatternChart`, де вісь Y відображає години доби

від 00:00 до 24:00, а стовпці показують діапазон активного сну у кожен день, дозволяючи візуально виявляти порушення нічного режиму. Під графіком сну відображається підсумковий рядок із тривалістю денного та нічного сну за тиждень. Якщо середня кількість замін підгузків виходить за межі норми (менше 5 або більше 8 на добу), екран відображає відповідне попередження лікарського характеру. Екран статистики наведено на рисунку 3.4.

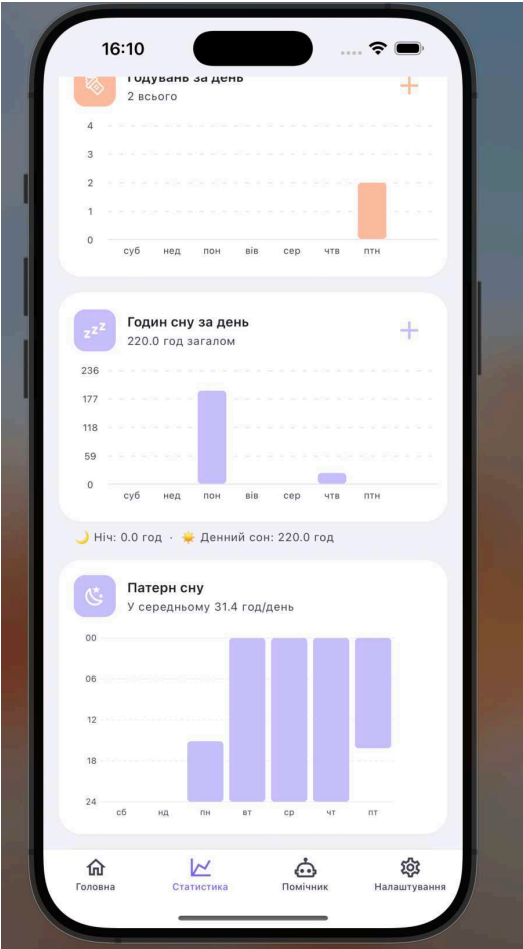


Рисунок 3.4 – Екран статистики за 7 днів

Екран статистики підтримує експорт даних у формат PDF через функцію `exportStatsPdf` у файлі `lib/pdf.ts`. Функція динамічно завантажує бібліотеки `expo-print` та `expo-sharing` через `require()` замість статичного імпорту, що дозволяє екрану статистики коректно завантажуватись у Expo Go, де ці нативні модулі відсутні. При активації експорту функція `buildHtml` генерує повноцінний HTML-документ із

CSS-стилями, який містить вітальну секцію з іменем дитини та датою, окремі картки для годувань, сну та підгузків із таблицями по днях, а також секцію антропометричних вимірювань. Бібліотека expo-print конвертує цей HTML у .pdf-файл засобами нативного рушія платформи (Print.printToFileAsync), а expo-sharing відкриває стандартний системний діалог передачі файлу (Sharing.shareAsync), через який користувач може зберегти PDF у Files, надіслати електронною поштою або передати через AirDrop. Всі текстові рядки та формати дат у PDF локалізовані відповідно до поточної мови застосунку через i18next та date-fns.

Форми введення даних побудовані на єдиному компоненті FormScreen, що реалізує стандартний макет із заголовком, кнопкою закриття та прокручуванням тілом з урахуванням клавіатури. Вибір типу активності реалізований через компонент KindGrid — сітку іконок-плиток, де кожен варіант відображається у вигляді картки з кольоровою іконкою та підписом, а активний елемент підсвічується кольоровою рамкою. Форма годування підтримує п'ять типів: ліву та праву груди, пляшку з грудним молоком, пляшку з сумішшю і тверду їжу, — а також два режими введення: запуск таймера в реальному часі кнопкою «Старт зараз» або ручне введення тривалості минулого сеансу. Форма підгузків підтримує чотири типи: мокрий (wet), брудний (dirty), змішаний (mixed) та сухий (dry), — що дозволяє точніше відстежувати стан дитини та виявляти можливі проблеми зі здоров'ям. Вибір дати і часу реалізований через DateField, що відкриває нативний системний пікер дати платформи, забезпечуючи звичний для користувача досвід. Форма додавання годування наведена на рисунку 3.6.

Екран III-асистента (app/(tabs)/assistant.tsx) реалізований як чат-інтерфейс із контекстом активної дитини. У верхній частині розміщена брендова картка з іменем та віком дитини у місяцях, що автоматично

передаються до Edge Function для формування персоналізованих відповідей. При порожній історії повідомлень відображаються чотири швидкі підказки (SUGGESTIONS_KEYS: sleep, feeding, milestone, teeth), натискання на які одразу надсилає відповідний запит без ручного введення. Під час очікування відповіді відображається індикатор завантаження. Поле вводу обмежено 400 символами (maxLength=400), а кнопка відправлення блокується під час виконання запиту. При наявності повідомлень у верхньому правому куті з'являється кнопка очищення історії чату поточної сесії.

Екран налаштувань (app/(tabs)/settings.tsx) організований у три секції. Секція «Профіль» надає доступ до редагування імені та email, а також до зміни пароля через відповідні модальні форми. Секція «Налаштування» містить перемикач мови (українська/англійська) та вибір теми оформлення. Секція «Акаунт» включає кнопки виходу та видалення облікового запису, кожна з яких відкриває ConfirmDialog для підтвердження деструктивної операції. Зміна мови відбувається реактивно через хук useLocale без перезавантаження застосунку. Інтерфейс екрана налаштувань наведено на рисунку 3.5.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						50
Зм.	Лист	№ докум.	Підп.	Дата		

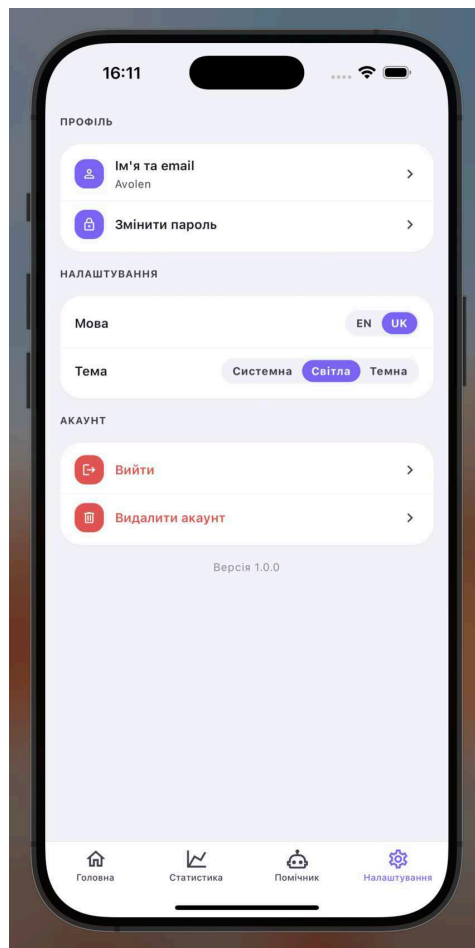


Рисунок 3.5 – Екран налаштувань застосунку

Екран вікових досягнень (`app/milestones/index.tsx`) відображає шаблони контрольних точок розвитку дитини, згрупованих за віковими діапазонами. Шаблони завантажуються з таблиці `milestone_templates` і збагачуються даними про фактичні досягнення дитини з таблиці `milestone_achievements`. Якщо вік дитини потрапляє у діапазон (`expected_age_min_months`–`expected_age_max_months`) шаблону, і досягнення ще не зафіксоване, шаблон потрапляє до секції «Актуально зараз» на початку екрана, привертаючи увагу батьків до поточних цілей розвитку. Кожен шаблон відображається як картка з категорійною іконкою, описом та статусом; натискання відкриває екран деталей, де можна зафіксувати дату досягнення та додати нотатку.

Екран вакцинацій (`app/vaccinations/index.tsx`) побудований

аналогічно до екрана досягнень, але групує щеплення за педіатричними слотами (при народженні, 2 місяці, 4 місяці, 6 місяців тощо). Шаблони завантажуються з таблиці `vaccination_templates` та зіставляються з фактичними записами у `vaccination_records` за полем `vaccine_code`. Секція «Актуально зараз» виокремлює вакцини, термін яких настав відповідно до поточного віку дитини. Кожна картка відображає назву вакцини, номер дози, групу та колірне кодування за типом щеплення; при натисканні відкривається форма фіксації дати введення та нотатки.

Редагування профілю дитини доступне через екран `app/children/[id]/edit.tsx`, що дозволяє змінити ім'я, дату народження, стать та нотатки, а також завантажити або видалити аватар через `expo-image-picker`. Видалення акаунту виконується з екрана налаштувань і вимагає підтвердження через `ConfirmDialog`, після чого каскадно видаляються всі пов'язані записи завдяки `ON DELETE CASCADE` у схемі бази даних.

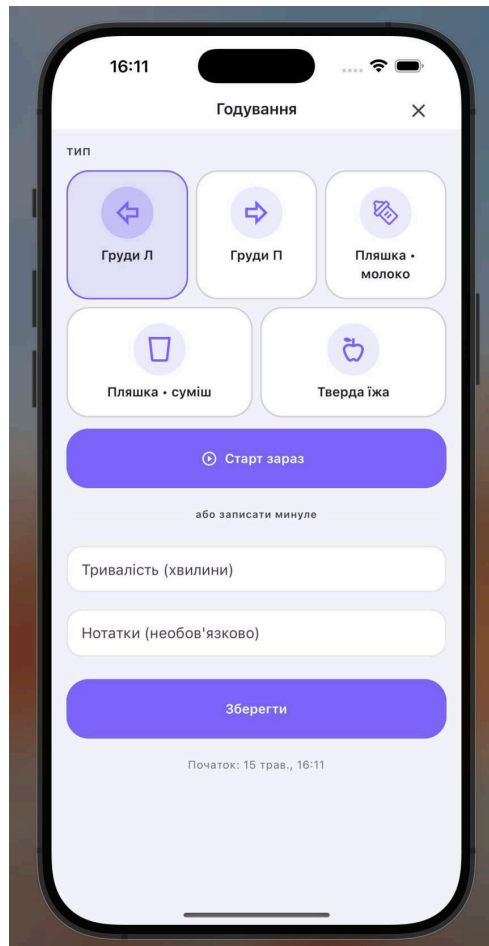


Рисунок 3.6 – Форма додавання запису годування

3.3.7 Робота з датами та часом

Уся робота з датами та часом у застосунку виконується бібліотекою `date-fns` — модульною утилітою, що надає функції для форматування, арифметики та порівняння дат без мутації об'єктів і без глобального стану. На відміну від `moment.js`, `date-fns` підтримує `tree-shaking`: до фінального пакету потрапляють лише ті функції, що реально імпортуються, що зменшує розмір збірки. Бібліотека використовується у трьох ключових контекстах. По-перше, у функції `formatAge` (`lib/age.ts`) для обчислення віку дитини: `differenceInDays`, `differenceInMonths` та `differenceInYears` визначають точний вік і обирають правильну одиницю відображення (дні, тижні, місяці або роки). По-друге, у функції `aggregateByDay` (`lib/aggregate.ts`) для формування 7-денних бакетів статистики: `subDays`

генерує масив дат від сьогодні назад, `parseISO` перетворює рядки з бази даних у об'єкти `Date`, а `isSameDay` зіставляє записи активностей із відповідним бакетом. По-третє, у модулі PDF-експорту (`lib/pdf.ts`) для локалізованого форматування дат у звіті через `format` із передачею об'єкта `locale` відповідно до поточної мови інтерфейсу.

3.3.8 Реалізація алгоритмів обробки медичних даних

Алгоритм класифікації антропометричних показників відповідно до стандартів ВООЗ реалізовано у файлі `features/measurements/who/index.ts`. Вихідні перцентильні таблиці зберігаються статично у `who/standards.ts` у вигляді вкладених числових масивів формату `[age, P3, P15, P50, P85, P97]`, де `age` — вік у місяцях окремо для кожної статі та кожного типу показника. Такий підхід дозволяє уникнути мережевих запитів при кожному розрахунку, оскільки таблиці компілюються безпосередньо у пакет застосунку та залишаються незмінними між сесіями. Стандарти ВООЗ охоплюють вікові діапазони від 0 до 24 місяців для дівчаток та хлопчиків окремо, що дозволяє враховувати статеві відмінності у нормах фізичного розвитку.

Функція `whoBands(sex, kind, ageMonths)` виконує лінійну інтерполяцію між двома суміжними рядками таблиці для точного вікового значення дитини. Спочатку обирається відповідна таблиця за статтю та типом показника, потім знаходяться два сусідні рядки, між якими знаходиться вік дитини (`floor(ageMonths)` та `ceil(ageMonths)`). Для кожного з п'яти перцентилів (`P3`, `P15`, `P50`, `P85`, `P97`) обчислюється інтерпольоване значення за формулою:

$$v = v_0 + (v_1 - v_0) \times (ageMonths - month_0),$$

де v_0 та v_1 — значення перцентилів для нижнього та верхнього рядків таблиці, $month_0$ — нижній цілий вік у місяцях. Функція повертає об’єкт BandValues з інтерпольованими значеннями п’яти перцентилів, що використовуються для побудови зон на графіку та класифікації вимірювання.

Функція `classifyValue(value, bands)` визначає, в яку зону потрапляє виміряне значення: нижче P3, між P3 і P15, між P15 і P85 (норма), між P85 і P97, або вище P97. Результати класифікації відображаються на графіку кольоровими зонами: зелена відповідає нормі між P15 та P85, жовті — граничним значенням P3–P15 і P85–P97, червоні — патологічним відхиленням нижче P3 та вище P97. Текстовий підпис під графіком дублює класифікацію словесно, що полегшує сприйняття для батьків без медичної освіти. Екран вимірювань із перцентильними зонами наведено на рисунку 3.7, екран хронологічної історії вимірювань — на рисунку 3.8.

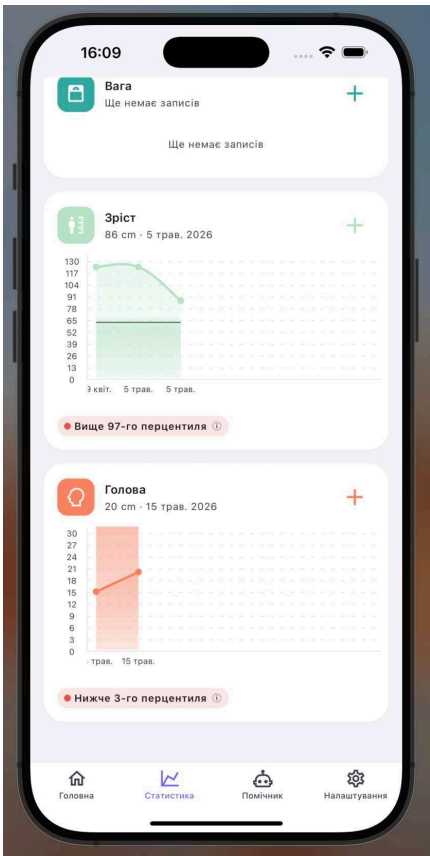


Рисунок 3.7 – Екран вимірювань із перцентильними зонами

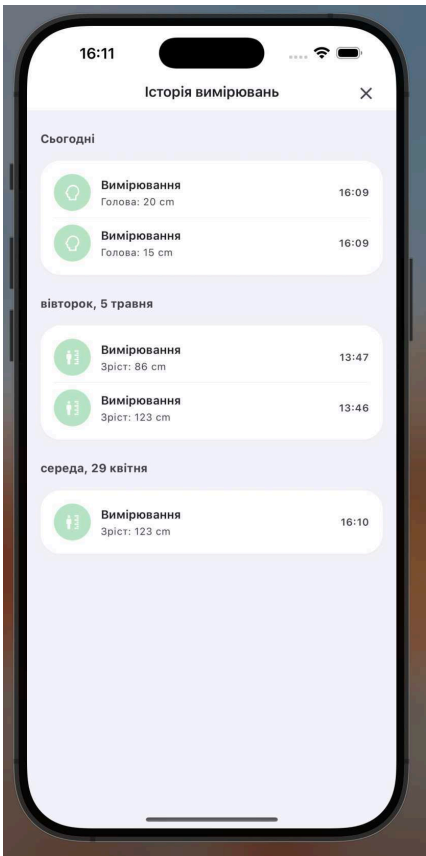


Рисунок 3.8 – Екран історії вимірювань

Алгоритм агрегації даних сну реалізовано у хуку useSleepPattern. Хук отримує список сесій сну за 7 днів і для кожного дня обчислює два показники: тривалість денного та нічного сну. Нічним вважається сон, що частково або повністю знаходиться в проміжку 19:00–06:59 наступного дня. Тривалість частини кожного сну, що перетинається з нічним вікном, обчислюється як довжина перетину двох часових відрізків, після чого значення підсумовуються для кожного дня окремо. Отримані дані передаються до компонента SleepPatternChart, який будує двовимірне подання режиму сну, що дозволяє батькам оцінити регулярність та тривалість нічного відпочинку дитини протягом тижня і своєчасно виявити порушення режиму сну.

ВИСНОВОК ДО РОЗДІЛУ 3

У третьому розділі розглянуто реалізацію кросплатформної клієнт-серверної системи агрегації та обробки медико-біологічних даних.

Спроековано та реалізовано архітектуру системи, що включає клієнтський застосунок на базі React Native та Expo з feature-based організацією коду і хмарний бекенд на платформі Supabase. Відокремлення шару представлення від шару даних та зосередження логіки авторизації на рівні СУБД забезпечують підтримуваність коду та унеможливлюють витік даних між обліковими записами незалежно від логіки клієнтського застосунку.

Реалізовано реляційну схему бази даних PostgreSQL з восьма предметними таблицями відповідно до вимог третьої нормальної форми. Для кожної таблиці визначено RLS-політики, що ізолюють дані різних користувачів на рівні СУБД без додаткової логіки авторизації в клієнтському коді. Реалізовано тригер автоматичного призначення власника при створенні нового профілю дитини, що скорочує кількість клієнтських запитів із двох до одного.

Розроблено Edge Function ask-assistant для III-асистента на базі RAG-архітектури з векторним пошуком у педіатричній базі знань через pgvector. Реалізовано систему автентифікації з підтримкою email/пароль та Google OAuth, а також синхронізацію між пристроями в реальному часі через Supabase Realtime із середньою затримкою 1–2 секунди.

На клієнтській частині реалізовано алгоритм класифікації антропометричних показників відповідно до стандартів BOOЗ з лінійною інтерполяцією перцентильних таблиць для точного вікового значення дитини, а також алгоритм аналізу структури сну з розподілом на денну та нічну складові. Розроблені алгоритми медично коректні та відповідають

офіційним рекомендаціям ВООЗ щодо оцінювання фізичного розвитку дітей.